Software Development 2

BSCH-SD2

Milestone 2 – Group D

“Campus Craze”

Date: 11/04//2026

Table of Contents

1. **Overview** 6

Game overview 6

Key concepts 6

Main Aim of finalized project 6

Technical Aims 7

Gameplay Aims 7

Team and Process Aims 7

Game rules: 8

2. **Milestone 1 details** 9

2.1 Development Process 9

2.2 Research 10

2.3 External Packages 11

2.4 Project Setup 11

2.5 Milestone 1 aims: 15

2.6 Achievements: 16

2.7 Issues: 20

2.8 Testing 22

Unit testing by Victoria Adeosun: 22

Unit testing by Paakhi Agrawala: 24

Unit testing by Emmanuel Ayobanjo: 25

Unit testing by Deliwe Mufute: 27

2.9 Commit Logs & Branches Tree 29

3. **Milestone 2 details** 32

3.1 Milestone 2 aims: 32

..... 34

3.2 Achievements: 35

3.3 Issues: 38
3.4 Identified Limitations 38
3.5 Design Considerations and Future Improvements 39
3.6 Game Loop and User Experience 39
3.7 Testing 40
Unit testing by Victoria Adeosun: 40
Unit testing by Paakhi Agrawala: 43
Unit testing by Emmanuel Ayobanjo: 47
Unit testing by Deliwe Mufute: 55
3.8 Commit Logs & Branches Tree 59
4. **Appendix:** 62
5. **References** 63

GRIFFITH COLLEGE DUBLIN

Student name:	Student 1: Miguel Cofré Student 2: Victoria Adeosun Student 3: Emmanuel Ayobanjo Student 4: Deliwe Mufute Student 5: Paakhi Agrawala				
Student number:	Student 1: 3165545 Student 2: 3171479 Student 3: 3173959 Student 4: 3172567 Student 5: 3167352				
Faculty:	Computing Science				
Course:	BSCH/BSCO/EXCH		Stage/year:	2	
Subject:	Software Development 2				
Study Mode:	Full time		Part-time	☑	
Lecturer Name:	Haseeb Younis				
Assignment Title:	SD2 Project - Final Documentation				
Date due:	13/04/2026				
Date submitted:	11/04/2026				

Plagiarism disclaimer:

I understand that plagiarism is a serious offence and have read and understood the college policy on plagiarism. I also understand that I may receive a mark of zero if I have not identified and properly attributed sources which have been used, referred to, or have in any way influenced the preparation of this assignment, or if I have knowingly allowed others to plagiarise my work in this way.

I hereby certify that this assignment is my own work, based on my personal study and/or research, and that I have acknowledged all material and sources used in its preparation. I also certify that the assignment has not previously been submitted for assessment and that I have not copied in part or whole or otherwise plagiarised the work of anyone else, including other students.

Signed: Miguel Cofré Arce **Date:** 11/04/2026

Signed: Victoria Adeosun **Date:** 11/04/2026

Signed: Emmanuel Ayobanjo **Date:** 11/04/2026

Signed: Deliwe Mufute **Date:** 11/04/2026

Signed: Paakhi Agrawala **Date:** 11/04/2026

Please note: Students **MUST** retain a hard / soft copy of **ALL** assignments as well as a receipt issued and signed by a member of Faculty as proof of submission.

1. Overview

Game overview

"Campus Craze" is a vertically scrolling arcade-style experience where the player controls a character moving left and right across the screen while the environment continuously scrolls downward, creating the illusion of forward movement. The objective is to survive as long as possible by avoiding obstacles that appear dynamically and increase in difficulty over time. Background tiles and decorative elements are recycled to create a seamless, infinite world, while a heads-up display (HUD) keeps track of the player's score and status. The game begins from a menu screen and transitions into active gameplay, where collision detection determines failure conditions and triggers a game over state.

Key concepts

- 2D vertical scroller (Camera from top)
- Player constantly "moves" forward (Y-axis)
- Player can move side-to-side (X-axis) and slightly forward/backward.
- Scenery scrolls down continuously to simulate forward movement.
- Obstacles appear randomly; player must avoid them.
- Score / distance tracked.

Main Aim of finalized project

The aim of this project is to design and develop a 2D vertically scrolling game using Java and JavaFX, applying object-oriented programming principles and collaborative software development practices. The project seeks to combine engaging gameplay mechanics with solid software architecture, creating a playable prototype while fostering team collaboration and technical skill development.

Technical Aims

- Implement a functional game using **object-oriented design**, with clearly defined classes such as Player, Obstacle, and GameScene.
- Apply **JavaFX** for rendering graphics, handling user input, and managing game scenes.
- Develop a **modular architecture**, allowing independent development and easy integration of game components.
- Implement **collision detection** and basic game physics to support core gameplay
- Integrate **unit testing** to validate core functionality and ensure code reliability.

Gameplay Aims

- Create a smooth **vertical scrolling system** that simulates continuous movement.
- Design and implement **dynamic obstacles** to challenge the player and enhance gameplay.
- Develop a **responsive player control system** using keyboard input for lateral movement.
- Include a **HUD** to display player score, lives, and other relevant information.
- Provide a simple **menu interface** to start and restart the game, including a game over screen.

Team and Process Aims

- Practice **version control using Git and GitLab**, including branching, merge requests, and collaborative development.
- Collaborate effectively using a **feature-branch workflow** to maintain code quality and organization.

- Manage integration through a **testing branch strategy**, ensuring features are combined systematically.
- Simulate a professional development environment with **code reviews, incremental delivery, and milestone-based assessment**.

Game rules:

The player controls a character running through a vertically scrolling city environment. The main objective is to survive as long as possible while avoiding obstacles such as pedestrians, construction equipment, and other hazards. The character can move **left and right** across the screen to dodge obstacles, but the forward motion is automatic, simulated by the continuously scrolling background and scenery tiles.

Players can collect **coins and special items** along the way to increase their score. Each obstacle collision or failure condition reduces the player's lives or ends the game, depending on the rules set for that milestone. The game keeps track of the **score** based on distance travelled and items collected, displayed on a **heads-up display (HUD)**.

Gameplay starts from a **menu screen**, where the player can start the game or view scores. Once the game ends, the **game over screen** is shown, and the player can return to the menu to try again. The game uses **randomized scenery and obstacle placement** to create an engaging and unpredictable challenge each time.

2. Milestone 1 details

2.1 Development Process

The project began with defining the concept of a 2D vertically scrolling game where the player avoids obstacles while the environment continuously moves. Based on this idea, the team identified key gameplay elements such as player movement, obstacles, and scoring. A decision was made on theme and then a prototype for the menu **(Figure 1)** and the game screen **(Figure 2)** was generated using Canva.



Figure 1: Main menu prototype



Figure 2: Game screen prototype

A Maven-based JavaFX project was then set up, and a GitLab repository was created with a structured branching strategy (`main`, `testing`, and feature branches) to support collaborative development. Find link to repository below:

https://gitlab.griffith.ie/miguelnicolas.cofrearce/sdii_groupproject

The system was designed using object-oriented principles, and core class skeletons were created, including Player, Obstacle, SceneryTile, GameScene, MenuScene, HUDManager, and AssetLoader.

Responsibilities were distributed among team members, allowing parallel implementation of components such as player movement, scenery scrolling, obstacles, and UI elements.

These components were integrated through merge requests into the testing branch, where functionality was verified. Basic unit tests were added to validate core features such as movement and collision detection. During this phase, issues identified through integration were resolved.

Finally, placeholder assets were created and incorporated to support visual testing, and documentation was maintained throughout the process.

2.2 Research

During the initial phase of the project, research was conducted to identify suitable technologies for developing a 2D game in Java. The team explored different graphical frameworks, including Swing (javax.swing) and JavaFX.

JavaFX was selected due to its modern features, improved graphics rendering capabilities, built-in support for animations, and better handling of user interface components compared to older frameworks such as Swing.

Additional research was carried out on JavaFX concepts such as **Nodes, Scenes, Panes, and the AnimationTimer**, which are essential for managing game loops and rendering elements on screen. Techniques for implementing **collision detection**, **keyboard input handling**, and **scene transitions** were also investigated to support core gameplay mechanics.

The team also reviewed best practices in **object-oriented design** and **game architecture**, ensuring that the project structure would allow modular development and easy integration of components.

2.3 External Packages

The project primarily relies on standard Java and JavaFX libraries, which provide the necessary tools for building the game.

- **JavaFX**: Used for creating the graphical user interface, handling scenes, rendering game elements (nodes), and managing user input. It also provides animation tools such as AnimationTimer, which is used to implement the game loop.
- **Maven**: Used as a build and dependency management tool to structure the project and handle required libraries efficiently.
- **JUnit (JUnit 5)**: Used for implementing unit tests to validate core functionality such as player movement, collision detection, and object initialization.

No third-party game engines or external APIs were used.

2.4 Project Setup

The project follows a standard Maven directory structure to ensure clarity and maintainability **(Figure 3)**. Main code and unit testing has been separated into specific folders. An extra folder for assets (“resources”) has been added as well, with nested folders for different types of assets.

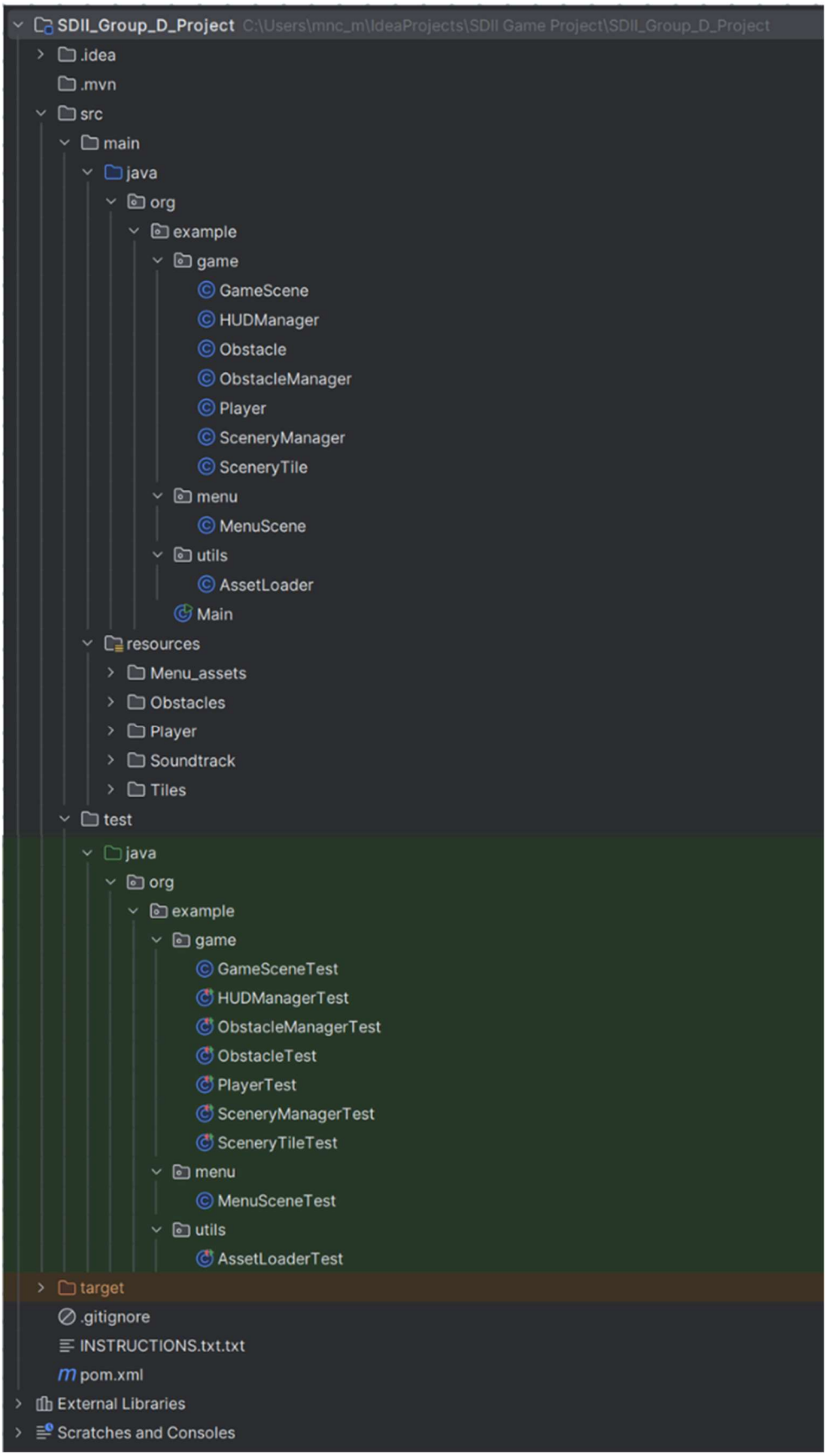


Figure 3: Project folder organization

An initial Class diagram was generated to roughly describe the project structure and to facilitate coding at the beginning of the project **(Figure 4)**.

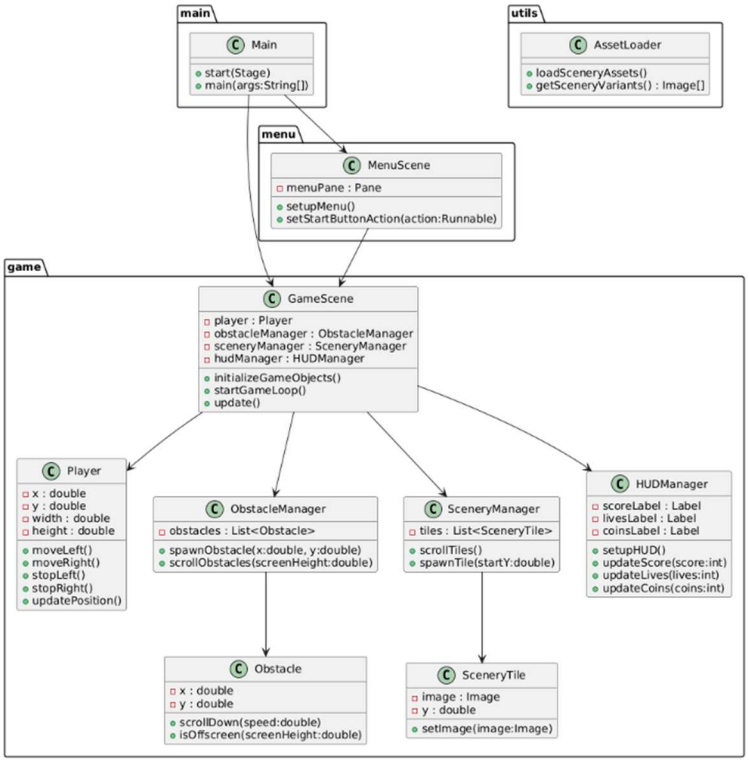


*Figure 4:*Initial Class diagram

This diagram was slowly updated until it reflected the current state of the code for Milestone 1 **(Figure 5).** This is planned to be updated with each new change or feature if needed.

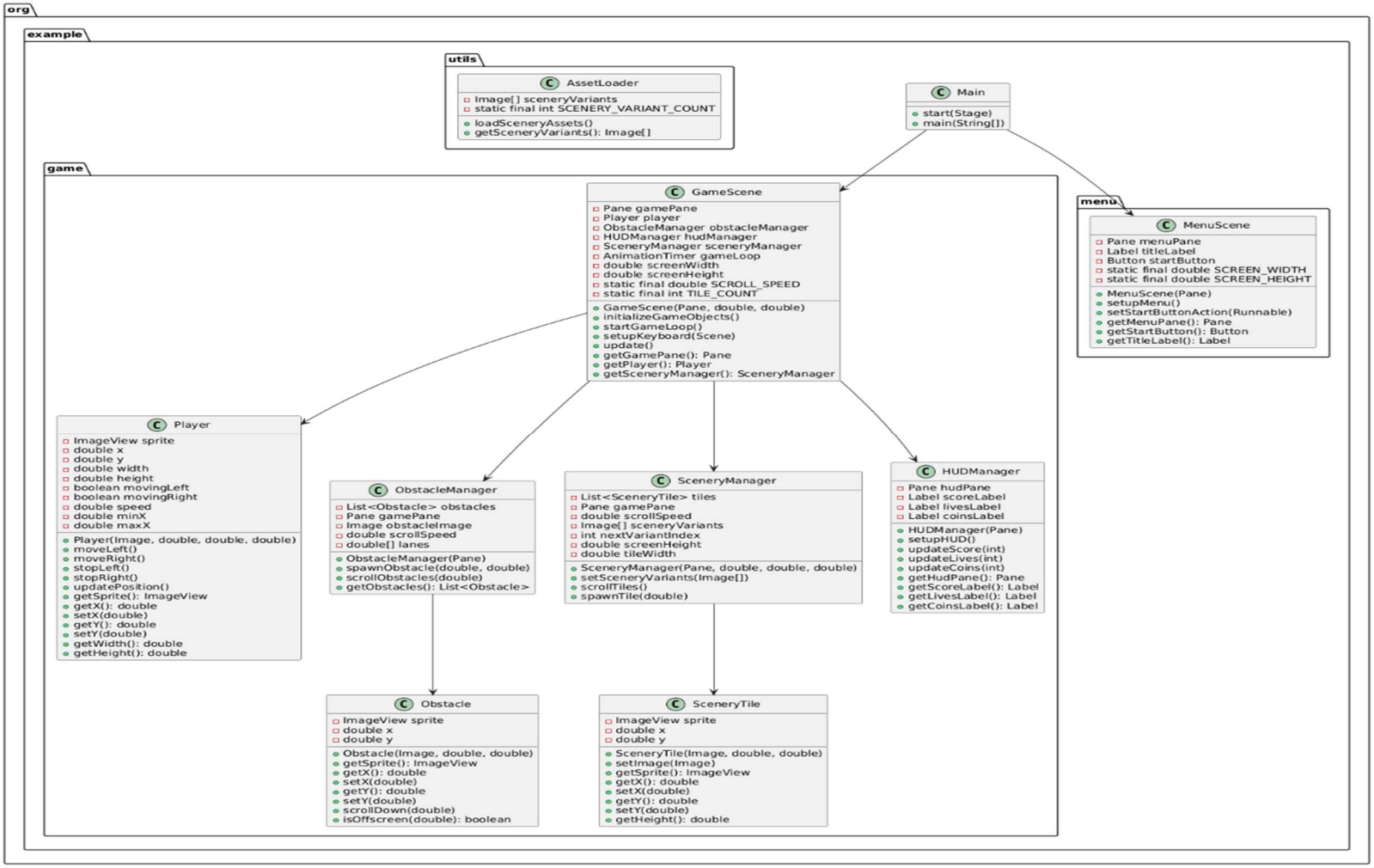


Figure 5: Updated Class Diagram for Milestone 1

2.5 Milestone 1 aims:

The following features were expected and checked for completion for Milestone 1:

Feature	Added
Basic player object with left/right movement controlled via keyboard input	OK
Scrolling scenery system using placeholder tiles to simulate vertical movement	OK
Initial static/dummy obstacles	OK
Start menu with a Start button to launch gameplay.	OK
HUD placeholders for **score and lives**, visible during gameplay	OK
Proper **JavaFX project structure** with packages for game, menu, and utils	OK
Skeleton **classes** for all main components: Player, Obstacle, SceneryTile, GameScene, MenuScene, HUDManager, AssetLoader.	OK
Feature branches for each module, merged into **testing branch** for integration	OK

2.6 Achievements:

All planned features for Milestone 1 were achieved; details of each member contributions are displayed on the tables below:

Miguel Cofré – Project Setup & Assets		
• Set up Maven/JavaFX project with correct pom.xml dependencies.		☑
• Confirm project builds and runs on everyone's machine.		☑
• Define package structure:		☑
	src/main/java/org/example/game/	☑
	src/main/java/org/example/menu/	☑
	src/main/java/org/example/utils/	☑
• Create class skeletons for all main classes:		☑
	Player.java	☑
	Obstacle.java	☑
	SceneryTile.java	☑
	GameScene.java	☑
	MenuScene.java	☑
	HUDManager.java (placeholders)	☑
	AssetLoader.java	☑
• Prepare assets:		☑
	Placeholder images: player sprite, obstacles, scenery tiles	☑
	Placeholder audio (BGM)	☑
	Test loading assets into memory using AssetLoader (images and audio preloaded).	☑
• Push project structure and assets to the testing branch.		☑

Deliwe Mufute – Player Class & Movement		
• Implement Player class:		☑
• Include ImageView for sprite.		☑
• Include position properties (layoutX, layoutY).		☑
• Implement keyboard input handling:		☑
	Left/Right movement only (X-axis).	☑
	Use Scene.setOnKeyPressed() / setOnKeyReleased().	☑
• Create methods:		☑
	moveLeft()	☑
	moveRight()	☑
	updatePosition()	☑
• Integrate Player with gamePane in GameScene:		☑
	Ensure sprite appears at correct starting position.	☑
	Ensure player node is hidden before pressing Start button.	☑
• Test basic movement on placeholder scenery.		☑

Emmanuel Ayobanjo – SceneryManager & Scrolling		
• Implement SceneryTile class:		☑
	Holds ImageView for each tile.	☑
• Implement SceneryManager class:		☑
	Manage list of tiles for vertical scrolling.	☑
• Methods:		☑
	scrollTiles()	☑
	spawnTile()	☑
• Load all 16 scenery variants as preloaded assets.		☑
• Implement recycling of offscreen tiles for performance:		☑
	Move tile back to top when it goes offscreen.	☑
• Integrate SceneryManager into GameScene.		☑
• Test scrolling during dummy game loop.		☑

Paakhi Agrawala – ObstacleManager & Dummy Obstacles		
• Implement Obstacle class:		☑
• Holds ImageView, position properties.		☑
• Implement ObstacleManager class:		☑
	Manage static/dummy obstacles:	☑
	Create initial obstacles in starting positions.	☑
• Methods:		☑
	spawnObstacle()	☑
	scrollObstacles()	☑
• Integrate ObstacleManager into GameScene.		☑
• Ensure obstacles move with scenery (scrolling).		☑
• Test visibility and correct alignment on gamePane.		☑

Victoria Adeosun – UI / Menu / HUD		
• Implement MenuScene class:		☑
	Display title label.	☑
	Add Start button.	☑
	Attach button event to call startGame()	☑
• Implement HUD placeholders:		☑
	Labels for Score, Coins, Lives.	☑
	Display during gameplay after Start pressed.	☑
• Ensure the menu disappears when the Start button is clicked.		☑
• Test basic switch from menu to gamePane.		☑

The game overall presents an incomplete state at the moment but with expected functionalities already in place. Current state of the menu **(Figure 6)** and main game screen running **(Figure 7)** can be found below.



Figure 6: Main menu screen state for Milestone 1

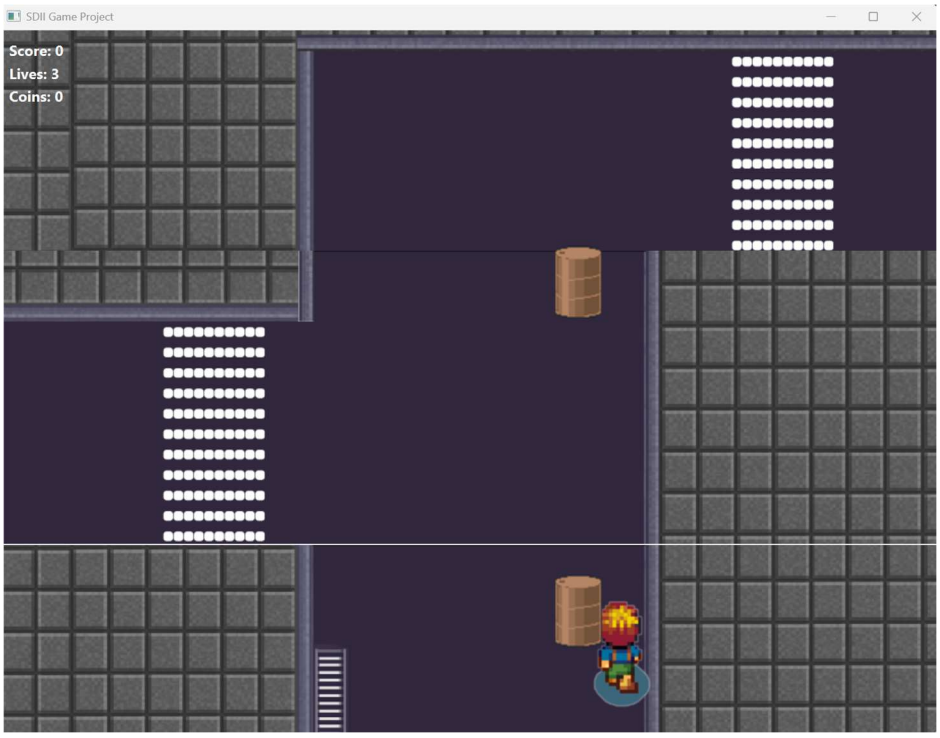


Figure 7: Main game running screen for Milestone 1

2.7 Issues:

During Milestone 1, several challenges were encountered while developing the project:

- **Branching and Merge Conflicts**
  - Some team members accidentally merged feature branches directly into the testing branch or attempted merges without pushing their changes, causing empty or redundant merge requests.
  - **Resolution:** A clear branching workflow was reinforced, emphasizing that each feature branch should be merged via merge request only, and team members must pull the latest testing branch before starting new work.
- **Coordination Between Team Members**
  - Since multiple members were working on interconnected systems (e.g., Player, ObstacleManager, SceneryManager), there was initial uncertainty about how to implement interactions without conflicting code.
  - **Resolution:** We established clear attribute placeholders in the main classes and used a shared understanding of object interactions. Regular communication and analogies (e.g., the "puzzle pieces" approach) helped reduce hesitation and confusion.
- **Final vs. Modifiable Attributes**
  - Some constants, such as SCREEN_WIDTH and SCREEN_HEIGHT, were initially declared as final, limiting flexibility when passing screen dimensions from Main.
  - **Resolution:** These were refactored to instance variables and passed through constructors, allowing adjustable resolution and improving integration across classes.
- **Unit Testing Awareness**

- Team members were unsure which methods or classes required unit testing and initially started writing tests post-integration. This delayed test development and highlighted the difference between testing for compliance versus testing for functionality.
- **Resolution:** A plan was made for future milestones to write unit tests **before** integration, ensuring tests verify meaningful behavior rather than just satisfying course requirements.

- **Asset Loading and Resource Paths**
  - Preloading images from the resources folder occasionally failed due to missing or misnamed files.
  - **Resolution:** Implemented error handling in AssetLoader to use fallback placeholders when resources were unavailable, preventing runtime crashes.
- **Keyboard Input Handling**
  - Integration of keyboard input with the Player class required careful handling to avoid null pointer exceptions when objects were not yet initialized.
  - **Resolution:** Checks were added in input handlers to ensure that only initialized objects receive commands, preventing runtime errors.
- **Deviation from Asset Usage in Menu**
  - In one case, the start menu was implemented using CSS-styled buttons and backgrounds instead of the planned asset images.
  - **Resolution:** This did not cause functional conflicts with other modules, and the game still ran correctly. It highlighted a minor misalignment between initial design specifications and actual implementation. While

not critical, it served as a reminder to verify that agreed-upon resources are used consistently across the team.

Overall, the challenges highlighted the importance of **team coordination, clear branching workflows, and early planning for unit tests**, which will be applied in future milestones to improve efficiency and reduce integration issues.

2.8 Testing

Unit testing by Victoria Adeosun:

- **HUDManager Tests:**
  - **testScoreLabelInitialText:** checks that the score label object is created and not null
  - **testScoreLabelStartsAtZero:** checks the score label shows "Score: 0" when the game starts
  - **testUpdateScoreChangesLabel:** calls updateScore(100) and checks the label actually changed
  - **testUpdateScoreMultipleTimes:** calls updateScore 3 times and checks it always shows the latest
  - **testUpdateScoreResetsToZero:** checks the score can go back to 0 after being updated
  - **testLivesLabelIsNotNull:** checks the lives label object is created and not null
  - **testLivesLabelStartsAtThree**: checks the lives label shows "Lives: 3" when the game starts
  - **testUpdateLivesChangesLabel:** calls updateLives(2) and checks the label actually changed

- **testUpdateLivesZero:** calls updateLives(0) and checks the game over state shows correctly
- **testHudPaneIsStored:** checks the pane passed into HUDManager is stored correctly
- **testScoreUpdateDoesNotAffectLives:** updates score and checks lives label didn't change
- **testLivesUpdateDoesNotAffectScore**: updates lives and checks score label didn't change

- **MenuScene Tests:**
  - **testMenuPaneIsNotNull:** checks the menu pane is created and not null
  - **testMenuPaneIsSameReference:** checks the pane passed into MenuScene is the same one stored
  - **testStartButtonIsNotNull:** checks the start button is created and not null
  - **testMenuVisibleByDefault:** checks the menu is visible when the game first loads
  - **testMenuHidesWhenStartFired:** fires the start button and checks the menu hides
  - **testStartButtonTriggersStartGame:** fires the start button and checks startGame() was called
  - **testGameNotStartedBeforeButtonClick:** checks the game hasn't started before anything is clicked.

Unit testing by Paakhi Agrawala:

- **Obstacle Tests:**
  - **constructorSetsPositionAndSpritePropertiesCorrectly():** Tests that the obstacle is initialised with correct position and sprite properties.
  - **setXUpdatesLogicalXAndSpriteX():** Tests that updating X position correctly updates both logical and sprite coordinates.
  - **setYUpdatesLogicalYAndSpriteY():** Tests that updating Y position correctly updates both logical and sprite coordinates.
  - **scrollDownMovesObstacleVerticallyByGivenSpeed():** Tests that the obstacle moves vertically according to the given scroll speed.
  - **isOffscreenReturnsFalseWhenObstacleIsWithinScreenHeight():** Tests that obstacle is correctly identified as on-screen when within bounds.
  - **isOffscreenReturnsTrueWhenObstacleIsPastScreenHeight():** Tests that obstacle is correctly identified as off-screen when beyond screen height.
- **ObstacleManager Tests:**
  - **spawnObstacleAddsObstacleToList():** Tests correct addition of a new obstacle to the internal obstacles list.
  - **spawnObstacleAddsObstacleSpriteToPane():** Tests that the obstacle's sprite is correctly added to the game pane.
  - **spawnObstacleCreatesObstacleWithCorrectInitialValues():** Tests that a spawned obstacle has the correct initial X and Y coordinates.
  - **scrollObstaclesMovesObstacleDownByScrollSpeed():** Tests that obstacles move vertically according to the defined scroll speed.

- **scrollObstaclesRespawnsObstacleWhenItGoesOffscreen():** Tests that obstacles are repositioned correctly when they move offscreen.
- **respawnAlsoUpdatesSpriteCoordinatesCorrectly():** Tests that sprite coordinates are updated correctly after obstacle respawn.
- **scrollObstaclesHandlesMultipleObstacles():** Tests correct behaviour when multiple obstacles are managed and updated simultaneously.

Unit testing by Emmanuel Ayobanjo:

- **SceneryTileTests:**

Tests the SceneryTile class, which holds a single ImageView and tracks its position in the scrolling background.

- **testInitialPosition():** Tests that the X and Y coordinates are correctly assigned when a SceneryTile is constructed.
- **testSetXUpdatesSpriteX():** Tests that calling setX() updates both the internal field and the sprite's rendered X position on screen.
- **testSetYUpdatesSpriteY():** Tests that calling setY() updates both the internal field and the sprite's rendered Y position on screen.
- **testGetSpriteNotNull():** Tests that the ImageView sprite is never null after construction — a null sprite would cause a NullPointerException during rendering.
- **testSetImageSwapsImage():** Tests that setImage() correctly replaces the displayed image on the sprite, which is required for tile recycling.
- **testGetHeightReturnsImageHeight():** Tests that getHeight() returns a positive value when a valid image is loaded, used by SceneryManager for offscreen detection.

- **testGetHeightFallbackWhenImageNull():** Tests that getHeight() returns the default fallback value of 192 when the tile's image is null, preventing division-by-zero errors during tile stacking.

- **SceneryManagerTest**

Tests the SceneryManager class, which manages the list of SceneryTile objects and drives the vertical scrolling background effect.

- **testInitialTileListIsEmpty():** Tests that the tile list starts empty before any tiles are spawned.
- **testSpawnTileAddsTileToList():** Tests that calling spawnTile() adds exactly one tile to the manager's internal tile list.
- **testSpawnTileAddsSpriteToPane():** Tests that the spawned tile's sprite is added to the JavaFX Pane, making it visible in the game scene.
- **testSpawnMultipleTiles():** Tests that spawning three tiles results in exactly three tiles tracked in the list.
- **testScrollTilesMovesTilesDown():** Tests that calling scrollTiles() moves all tiles to a lower Y position, confirming downward scroll direction.
- **testScrollTilesMovesDownByScrollSpeed():** Tests that each scrollTiles() call moves tiles by exactly the configured scroll speed value — verifies the scroll rate is precise.
- **testOffscreenTileIsRecycledToTop():** Tests that a tile which moves below screenHeight is recycled back above the screen (negative Y), implementing the continuous scroll effect.
- **testTileCountUnchangedAfterRecycle():** Tests that recycling reuses the existing tile object rather than creating a new one confirms no unnecessary object creation during gameplay.

- **testGetScrollSpeed():** Tests that getScrollSpeed() returns the correct value set at construction.
- **testSetScrollSpeed():** Tests that setScrollSpeed() updates the scroll rate correctly, allowing speed changes during gameplay.
- **testSpawnWithNoVariantsDoesNotCrash():** Tests that spawnTile() handles an empty variants array gracefully without throwing an exception defensive test for misconfiguration.

- **AssetLoaderTest**
  - **testGetSceneryVariantsNullBeforeLoad():** Tests that getSceneryVariants() returns null before loadSceneryAssets() is called, confirming no premature loading occurs.
  - **testLoadSceneryAssetsCreatesArray():** Tests that calling loadSceneryAssets() initialises the variants array so it is no longer null.
  - **testLoadSceneryAssetsReturns16Slots():** Tests that the returned array always contains exactly 16 slots regardless of missing asset files, matching the required scenery variant count.
  - **testNoSlotThrowsException():** Tests that loadSceneryAssets() does not throw an exception when asset files are missing — the fallback logic should handle missing files silently.

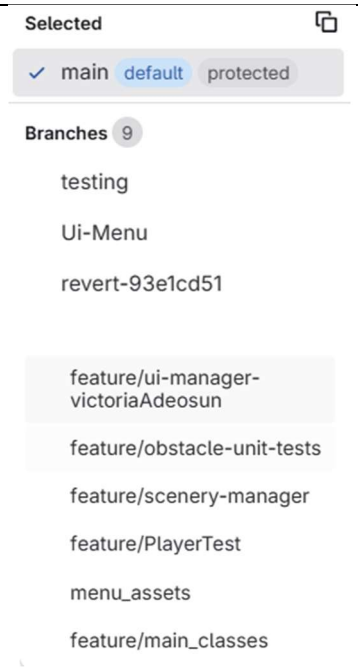
Unit testing by Deliwe Mufute:

- **testInitialPosition():** Verifies that the player is correctly initialised with the given starting X and Y coordinates. This ensures the constructor properly assigns position values.

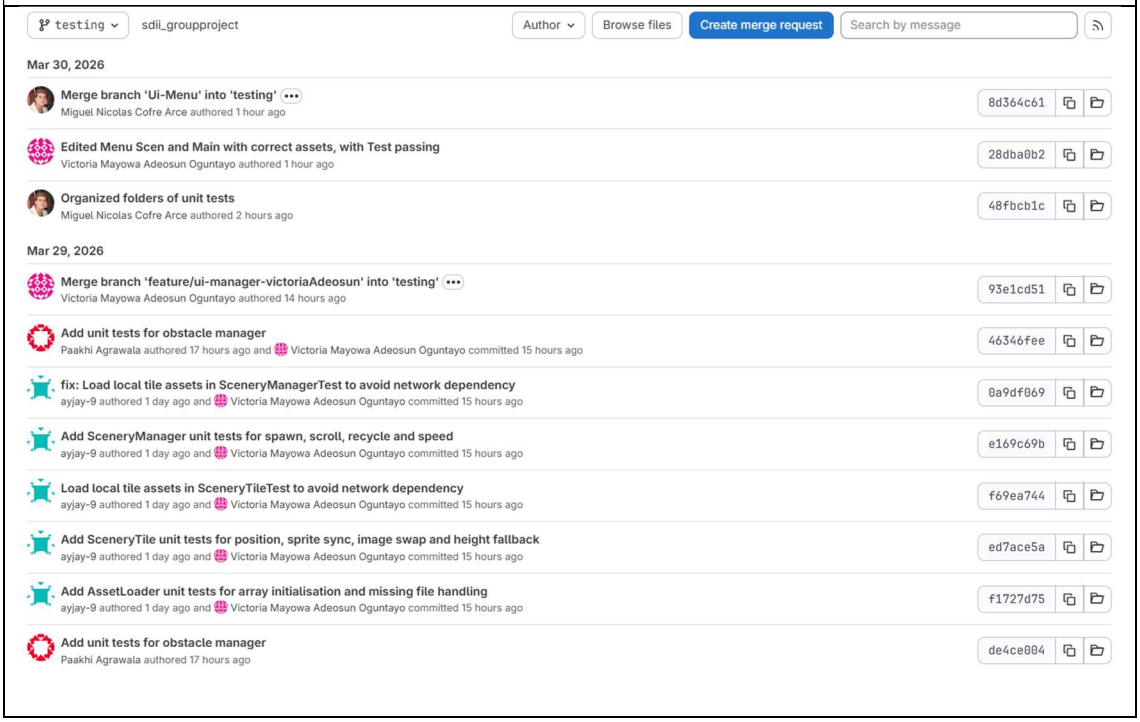
- **testMoveLeft():** Tests that when moveLeft() is triggered and updatePosition() is called, the player's X position decreases. This confirms left movement logic works correctly
- **testMoveRight():** Tests that when moveRight() is triggered and updatePosition() is called, the player's X position increases. This confirms right movement logic works correctly.
- **testStopLeft():** Ensures that after calling stopLeft(), the player no longer moves left when updatePosition() is executed. This validates correct handling of movement stopping logic.
- **testStopRight():** Ensures that after calling stopRight(), the player no longer moves right when updatePosition() is executed. This validates correct stopping behaviour for right movement.
- **testLeftBoundary():** Checks that the player cannot move beyond the left edge of the screen (X = 0). This ensures boundary constraints are enforced.
- **testRightBoundary():** Checks that the player cannot move beyond the right edge of the screen (screen width minus player width). This ensures the player remains within visible bounds.
- **testUpdatePositionSyncsSprite():** Verifies that after updating position, the ImageView sprite coordinates match the player's internal X and Y values. This ensures visual representation stays consistent with logic.

2.9 Commit Logs & Branches Tree

Branches in repository:



Commit Logs



Mar 28, 2026

- Merge branch 'feature/scenery-manager' into 'testing' — Deliwe Mufute authored 1 day ago — cc9b549e
- fix: Load local tile assets in SceneryManagerTest to avoid network dependency — ayjay-9 authored 1 day ago — 699593f2
- Add SceneryManager unit tests for spawn, scroll, recycle and speed — ayjay-9 authored 1 day ago — 8d2fed71
- Load local tile assets in SceneryTileTest to avoid network dependency — ayjay-9 authored 1 day ago — 667203e3
- Add SceneryTile unit tests for position, sprite sync, image swap and height fallback — ayjay-9 authored 1 day ago — 1e72e67f
- Add AssetLoader unit tests for array initialisation and missing file handling — ayjay-9 authored 1 day ago — 136dc290
- Merge branch 'testing' into 'feature/ui-manager-victoriaAdeosun' — Miguel Nicolas Cofre Arce authored 1 day ago — ad488fa8

Mar 27, 2026

- Merge branch 'feature/PlayerTest' into 'testing' — Emmanuel Ayobanjo authored 3 days ago — 99071053
- Initialise JavaFX toolkit in all test classes using PlatformImpl to resolve... — Deliwe Mufute authored 3 days ago — 787d23f9

Mar 26, 2026

- Add other test files — Deliwe Mufute authored 4 days ago and Emmanuel Ayobanjo committed 3 days ago — 91d7f199
- Merge branch 'menu_assets' into 'testing' — Victoria Mayowa Adeosun Oguntayo authored 3 days ago — 52dd4120
- Added HUDManager setupHUD with score, lives, coins labels - Victoria Adeosun — Victoria Mayowa Adeosun Oguntayo authored 3 days ago — 34780d75
- Implemented MenuScene, HUDManager setup and Junit test passing - Victoria Adeosun — Victoria Mayowa Adeosun Oguntayo authored 3 days ago — 259f0cfc

Mar 25, 2026

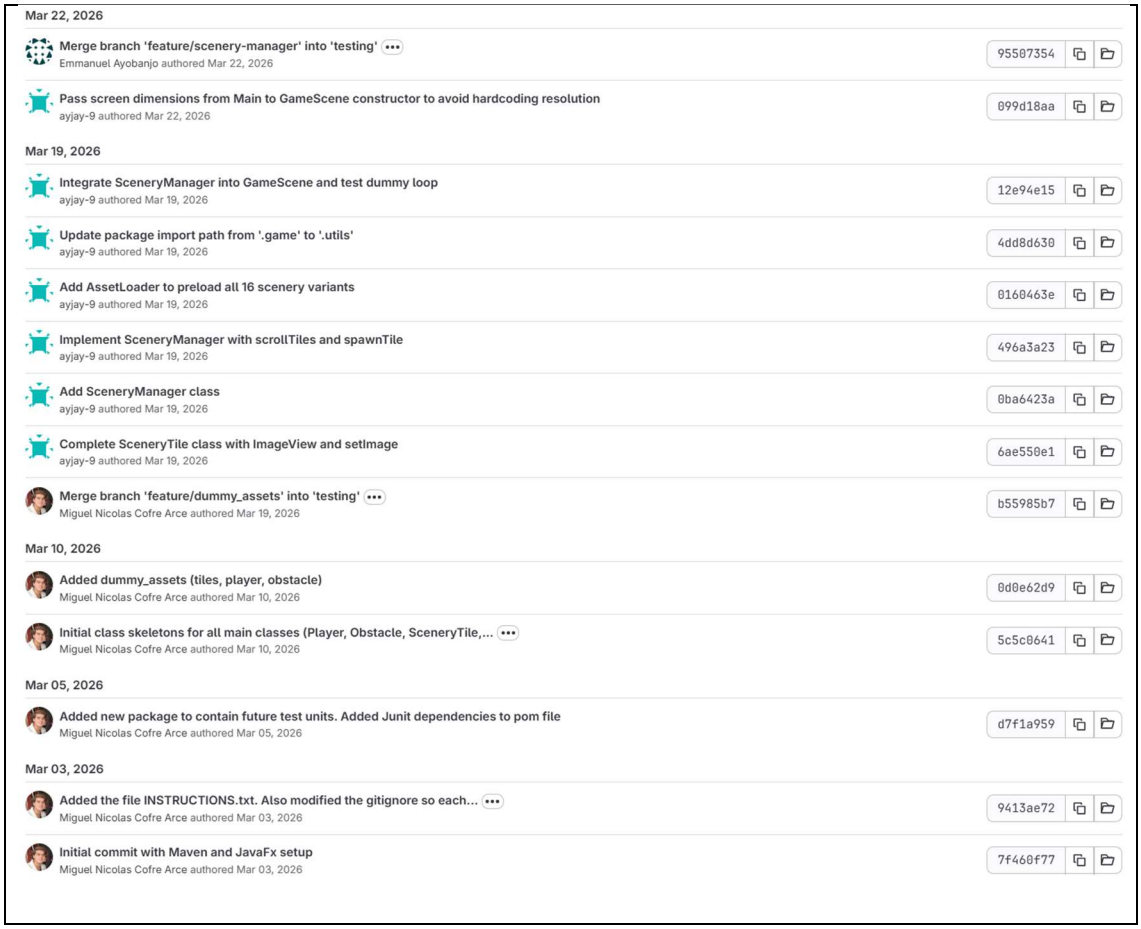
- Add other test files — Deliwe Mufute authored 4 days ago — fca9a1aa
- Add unit tests for Player movement, flags, and boundary constraints — Deliwe Mufute authored 4 days ago — fc30be59
- package org.example.game; — Deliwe Mufute authored 4 days ago — 6039d09e
- Added menu assets (background and buttons) — Miguel Nicolas Cofre Arce authored 4 days ago — 248eb188
- Add HUDManagerTest, Junit Test passing - Victoria Adeosun — Victoria Mayowa Adeosun Oguntayo authored 4 days ago — 24091a87
- Merge branch 'features/obstacles' into 'testing' — Miguel Nicolas Cofre Arce authored 5 days ago — c3f595b5
- Implemented obstacle system and integrated with game scene — Paakhi Agrawala authored 5 days ago — 484c6229

Mar 23, 2026

- Merge branch 'feature/Player_Class_and_Movements' into 'testing' — Emmanuel Ayobanjo authored 6 days ago — 43004869
- Pass screenWidth from Main to Player via GameScene to replace hardcoded maxX (1024) — Deliwe Mufute authored 6 days ago — 133382e9

Mar 22, 2026

- Connect keyboard input setup to GameScene — Deliwe Mufute authored 1 week ago — c0e2c5fe
- Add keyboard input handling for Player Movement — Deliwe Mufute authored 1 week ago — 183d66ca
- Update Player position in game loop — Deliwe Mufute authored 1 week ago — 4f2a772e
- Create Player object and add to GameScene — Deliwe Mufute authored 1 week ago — 0cd33408
- Update Player position with movement and boundary checks — Deliwe Mufute authored 1 week ago — 9b4b745d
- Implement Player movement methods (left and right) — Deliwe Mufute authored 1 week ago — 5f260150
- Add movement flags and speed variables to Player — Deliwe Mufute authored 1 week ago — c7510ecd


Mar 22, 2026
Merge branch 'feature/scenery-manager' into 'testing'
Emmanuel Ayobanjo authored Mar 22, 2026
95507354
Pass screen dimensions from Main to GameScene constructor to avoid hardcoding resolution
ayjay-9 authored Mar 22, 2026
099d18aa
Mar 19, 2026
Integrate SceneryManager into GameScene and test dummy loop
ayjay-9 authored Mar 19, 2026
12e94e15
Update package import path from '.game' to '.utils'
ayjay-9 authored Mar 19, 2026
4dd8d630
Add AssetLoader to preload all 16 scenery variants
ayjay-9 authored Mar 19, 2026
0160463e
Implement SceneryManager with scrollTiles and spawnTile
ayjay-9 authored Mar 19, 2026
496a3a23
Add SceneryManager class
ayjay-9 authored Mar 19, 2026
0ba6423a
Complete SceneryTile class with ImageView and setImage
ayjay-9 authored Mar 19, 2026
6ae550e1
Merge branch 'feature/dummy_assets' into 'testing'
Miguel Nicolas Cofre Arce authored Mar 19, 2026
b55985b7
Mar 10, 2026
Added dummy_assets (tiles, player, obstacle)
Miguel Nicolas Cofre Arce authored Mar 10, 2026
0d0e62d9
Initial class skeletons for all main classes (Player, Obstacle, SceneryTile,...
Miguel Nicolas Cofre Arce authored Mar 10, 2026
5c5c0641
Mar 05, 2026
Added new package to contain future test units. Added Junit dependencies to pom file
Miguel Nicolas Cofre Arce authored Mar 05, 2026
d7f1a959
Mar 03, 2026
Added the file INSTRUCTIONS.txt. Also modified the gitignore so each...
Miguel Nicolas Cofre Arce authored Mar 03, 2026
9413ae72
Initial commit with Maven and JavaFx setup
Miguel Nicolas Cofre Arce authored Mar 03, 2026
7f460f77

3. Milestone 2 details

3.1 Milestone 2 aims:

The following features were expected and checked for completion of Milestone 2:

Feature	Added
Integrate all modules; manage game state transitions (Menu → Gameplay)	OK
Expand Player: smooth side movement, slight forward/back movement, velocity limits	OK
Expand SceneryManager: 16 variants, random selection, infinite scrolling logic	OK
Expand ObstacleManager: dynamic spawning, movement, collision detection	OK
Implement ScoreManager: update HUD, scoring per distance/time, lives tracking	OK
Write basic automated tests: collisions, scrolling updates, input handling	OK
Merge feature branches, review code, prepare Milestone 2 documentation & individual reports	OK

To achieve this, the project architecture was re-designed following feedback from previous milestone. The previous class diagram was re-designed to represent a more Object-Oriented design in order to ease scalability and modularization of the project (**Figure 8**).

A second more detailed diagram was designed to ease team communication and to explain new architecture in more depth, expected new methods and attributes per class. This diagram was also colour coded to visually represent each member scope of work expected (**Figure 9**).

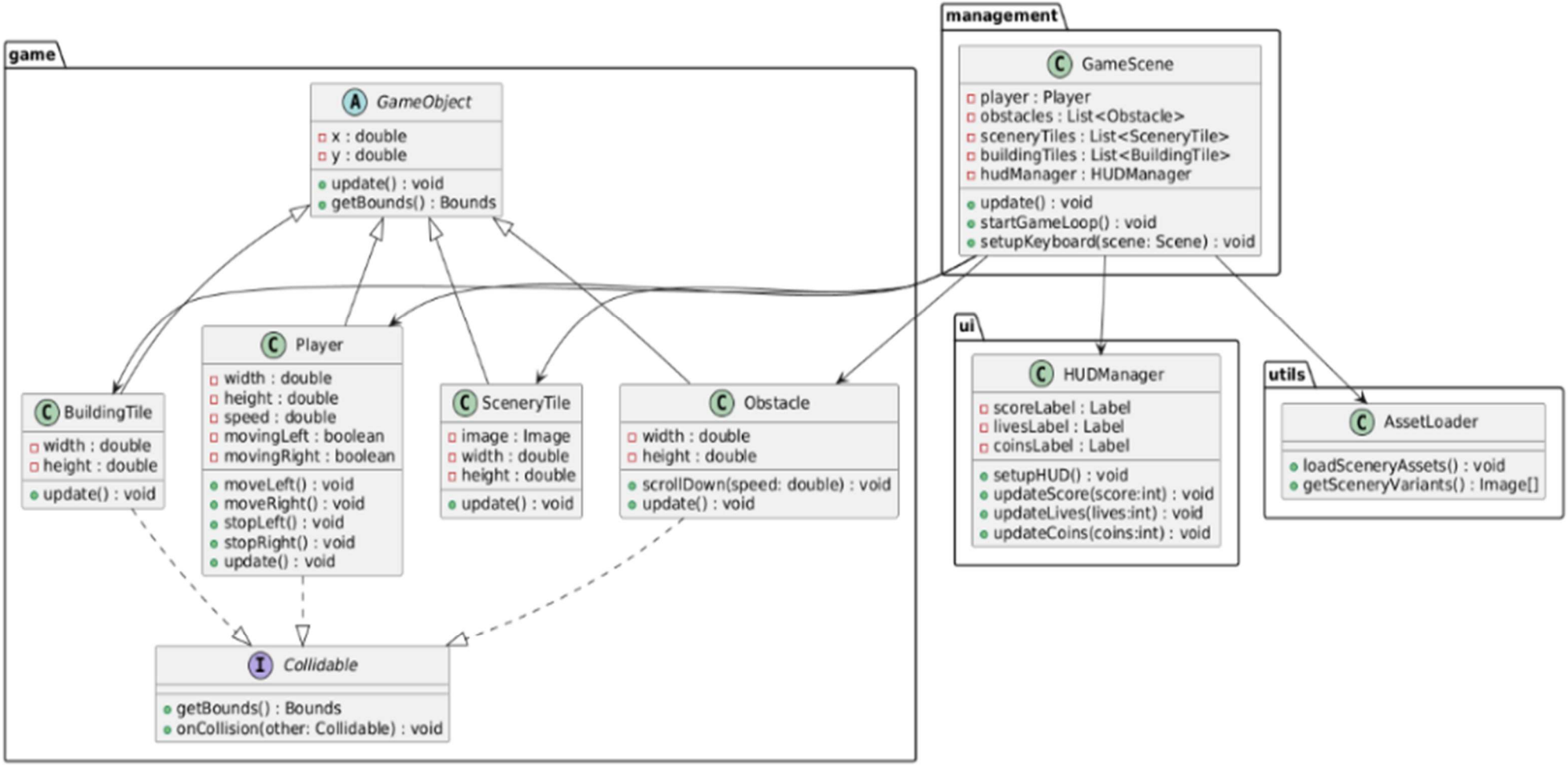


Figure 8: Class diagram displaying new project architecture for Milestone 2.

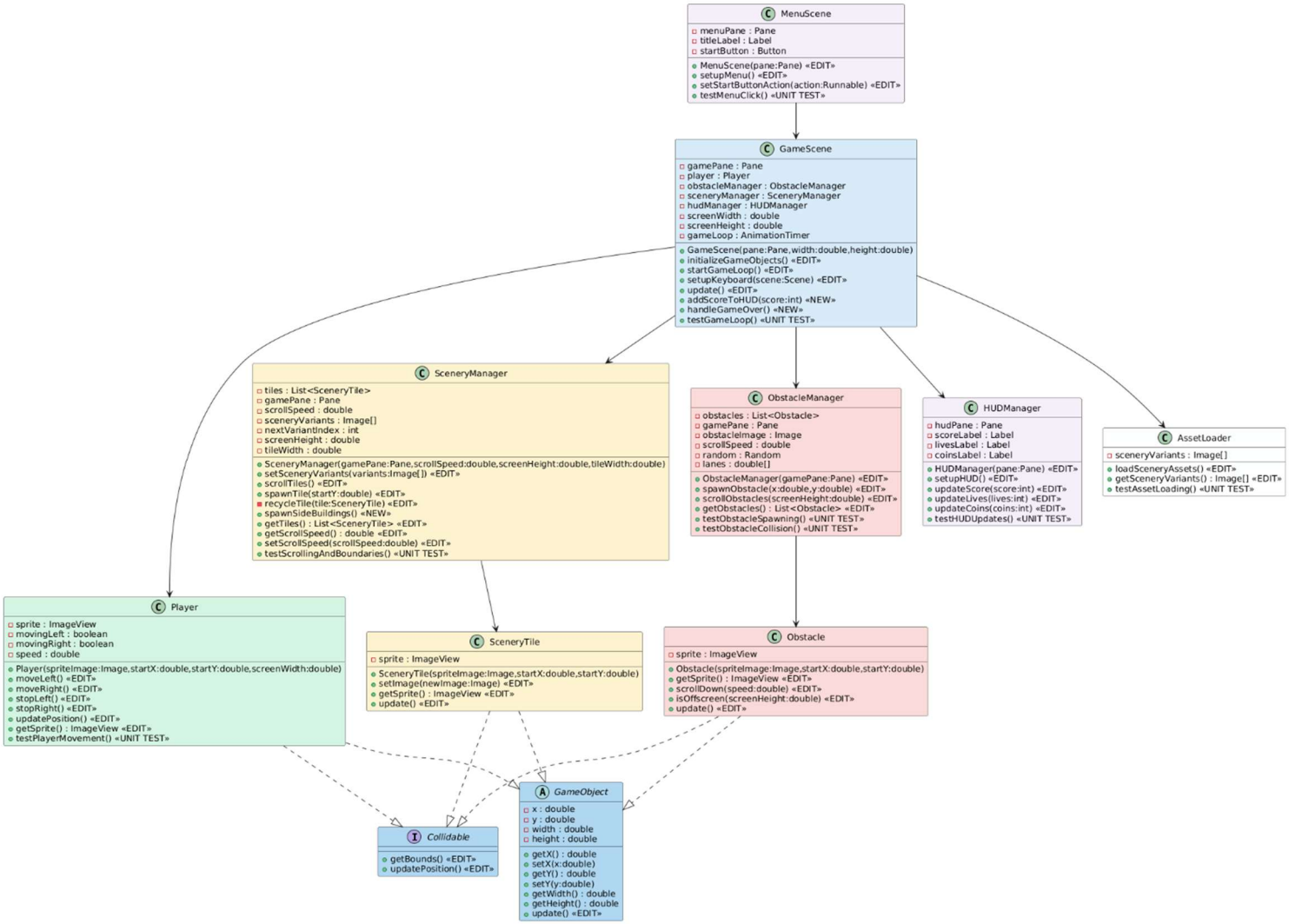


Figure 9: Detailed class diagram with methods expected and colour coding for workload distribution on Milestone 2.

3.2 Achievements:

All planned features for Milestone 2 were achieved; details of each member contributions are displayed on the tables below:

Miguel Cofré – Lead & Integration / OOP Architecture			
• OOP Design & Refactor			☑
	Create abstract class **GameObject**		☑
		Include fields Y and X	☑
		Methods **update(), getBounds()**	☑
	Create interface **collidable**		☑
		Methods **getBounds(), onCollision()**	☑
	Refactor **Player**, **Obstacle**, **SceneryTile** to inherit/implement		☑
	Refactor **GameScene** to store all game objects in **List<GameObject>**		☑
	Update game loop to polymorphically call **update()** on all objects		☑
	Handle **collision checks** using **collidable** interface		☑
• Integration			☑
	Merge all feature branches into testing		☑
	Ensure **keyboard input, HUD, and scoring** work		☑
	Maintain proper game state transitions: Menu → Gameplay → Game Over to be added on Milestone 3		☑
• Unit testing			☑
	Test polymorphic **update() loop in GameScene**		☑
	Test **integration of Player, Obstacles, SceneryTiles, Buildings**		☑
	Test **collision detection loop in GameScene**		☑

Deliwe Mufute – Player Class & Movement		
• OOP & Player Logic		☑
	Player extends GameObject	☑
	Implement **Collidable**	☑
	Smoothen left/right movement	☑
	Include slight forward and backward movement	☑
	Check collisions with Emmanuel's side buildings	☑
	Update **update()** method for movement and collisions	☑
• Unit testing		☑
	Test movement updates (left/right/forward/backward)	☑
	Test boundary constraints (screen edges and building collisions)	☑
	Test stop movement commands (**stopLeft(), stopRight()**)	☑

Emmanuel Ayobanjo – Scenery & Map Boundaries		
• SceneryTiles		☑
	SceneryTile extends GameObject	☑
	Implement **update()** for vertical scrolling	☑
	Infinite scroll using 16 variants (randomized)*	☑
• Side Buildings		☑
	buildingTile extends GameObject & implements **collidable**	☑
	Place buildings **randomly* on left/right sides**	☑
	Handle collisions with **Player**	☑
	Define playable area boundaries	☑
• Unit testing		☑
	Test **vertical scrolling updates**	☑
	Test **tile recycling logic**	☑
	Test **Player collision with side buildings**	☑
	Test **random* placement of building tiles**	☑

*I think we need to discuss randomization. I believe we could tie the variance to distance travelled? (time).

Paakhi Agrawala – Obstacles			
• Obstacle Logic			☑
	obstacle extends GameObject & implements **collidable**		☑
	Spawn obstacles dynamically		☑
	Add movable objects that move on each frame		☑
	Add Coins to be collected		☑
	Detect collisions with **Player (onCollision)**		☑
	Reposition offscreen obstacles randomly		☑
• Unit testing			☑
	Test **obstacle movement**		☑
	Test **collision detection with player**		☑
	Test **respawn logic**		☑

Victoria Adeosung – HUD & ScoreManager			
• HUD & Score			☑
	Keep HUDManager for score, lives, coins		☑
	Connect HUD updates with:		☑
		Distance traveled (or time)	☑
		Coins collected	☑
		Player lives (reduced on collision with obstacles)	☑
• Unit testing			☑
	Test **score increment logic**		☑
	Test **lives decrement on collisions**		☑
	Test **HUD updates after Player actions**		☑

At this stage of development, the core gameplay systems have been successfully implemented and stabilised. Obstacle spawning is functioning as intended, and collision detection between the player and obstacles is now correctly handled. The player loses a single life per collision event, ensuring consistent and predictable gameplay behaviour.

Additionally, the scenery system is operational, with background tiles scrolling correctly and supplementary environmental elements (such as side buildings) rendering as expected. These features collectively establish a functional gameplay loop that meets the primary requirements for this milestone. Current state of the gameplay screen running can be found below **(Figure 10)**.

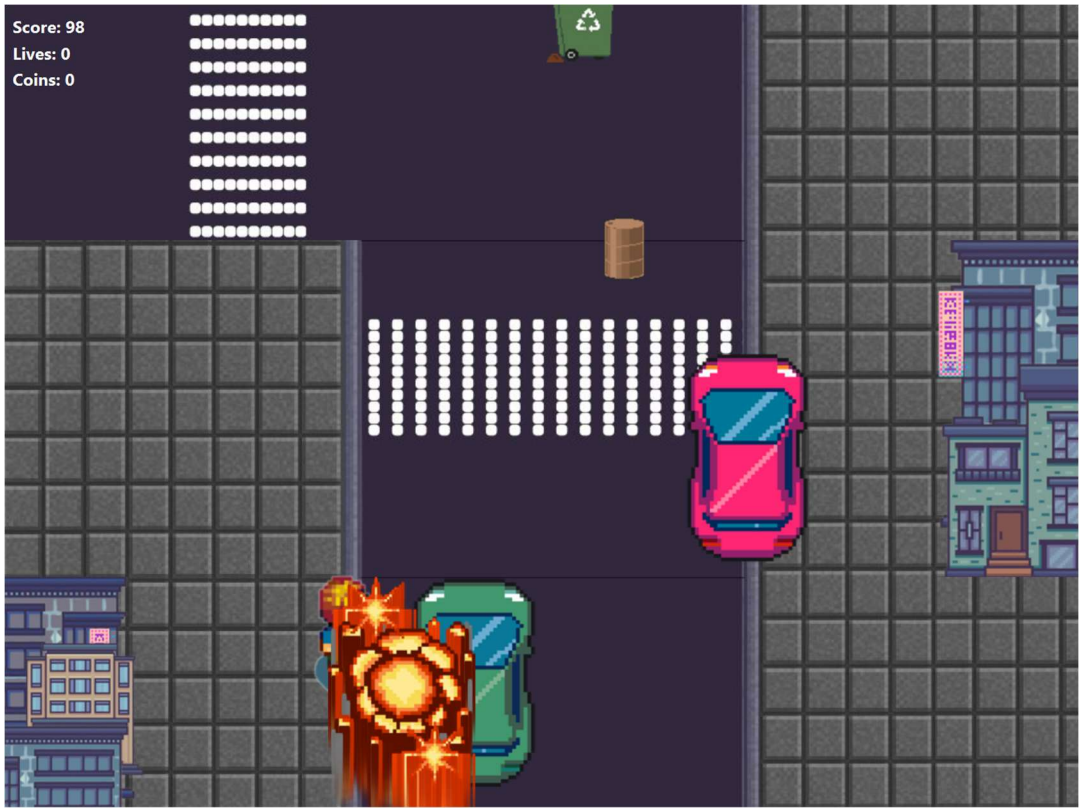


Figure 10: Current game screenshot for milestone 2.

3.3 Issues:

Issues encountered in Milestone 1 have been resolved successfully for Milestone 2. Currently the main concern stands in identified limitations on the state of the game so far.

3.4Identified Limitations

Despite the successful implementation of core mechanics, several limitations have been identified:

- **Lack of Theme-Based Variation:** Obstacle spawning is not yet context-aware. Obstacles designed for one environment (e.g., city) may appear in others (e.g., park or subway), reducing visual coherence.
- **Speed Inconsistency:** Obstacles currently move at a different speed than the background tiles, leading to a mismatch in perceived motion that affects gameplay realism.
- **Limited Player Movement Space:** The player's vertical movement is restricted to approximately one-third of the screen, limiting strategic manoeuvrability and reducing gameplay depth.
- **Obstacle Distribution Imbalance:** Obstacles tend to spawn centrally, allowing players to avoid them by remaining at the edges, which diminishes challenge.
- **Environmental Inconsistencies:** Certain objects (e.g., vehicles) persist across scene transitions where they are no longer contextually appropriate.

3.5Design Considerations and Future Improvements

Several improvements have been identified for future development:

- **Collision Tuning:** Introduce controlled leniency in collision detection (i.e., slightly forgiving hitboxes). This is a recognised game design technique that enhances perceived player skill and overall satisfaction[1].
- **Expanded Movement Freedom:** Increase the vertical range available to the player to encourage more dynamic obstacle avoidance.
- **Thematic Consistency:** Ensure that obstacle types are correctly matched to their respective environments to improve immersion.
- **Content Variation:** Introduce a wider variety of obstacles and environmental elements to reduce repetition and enhance visual engagement.

3.6Game Loop and User Experience

The current implementation lacks a complete game loop. Specifically:

- A Game Over screen has not yet been implemented and should include:
    - Final score
    - Performance metrics (e.g., number of collisions)
    - The main menu requires additional functionality, including:
        - An exit option

  - An introductory sequence displaying team and project information
  - Audio and Visual Polish
- Audio and feedback systems are not yet integrated. Future work should include:
  - Background music
  - Sound effects for collisions and interactions
  - Refinement of visual effects (e.g., explosion or impact feedback)

3.7Testing

Unit testing by Victoria Adeosun:

- # HUDManagerTest

Tests the HUDManager class, which manages the on-screen display labels for score, lives, and coins during gameplay. Also covers MenuScene construction and start button behaviour.

testScoreLabelInitialText() — Tests that the score label is not null after construction, confirming the HUD initialises correctly.

testScoreLabelStartsAtZero() — Tests that the score label displays "Score: 0" when the game first starts.

testUpdateScoreChangesLabel() — Tests that calling updateScore() correctly updates the label text to reflect the new value.

testUpdateScoreMultipleTimes() — Tests that calling updateScore() multiple times always shows the most recently set value.

testUpdateScoreResetsToZero() — Tests that the score label correctly resets back to "Score: 0" when updateScore(0) is called.

testScoreLargeValue() — Tests that the score label handles large values correctly without any display errors.

testLivesLabelIsNotNull() — Tests that the lives label is not null after construction.

testLivesLabelStartsAtThree() — Tests that the lives label displays "Lives: 3" at the start of the game.

testUpdateLivesChangesLabel() — Tests that calling updateLives() correctly updates the lives label text.

testUpdateLivesZero() — Tests that the lives label correctly shows "Lives: 0" at game over.

testLivesDecrementOnCollision() — Tests that lives decrease correctly by simulating two consecutive updateLives() calls.

testLivesReachZeroAtGameOver() — Tests that lives correctly reach zero when the game is over.

testHudPaneIsStored() — Tests that the HUD pane stored internally is the exact same pane passed into the constructor.

testScoreUpdateDoesNotAffectLives() — Tests that updating the score does not change the lives label.

testLivesUpdateDoesNotAffectScore() — Tests that updating lives does not change the score label.

testAllHUDLabelsUpdateIndependently() — Tests that score, lives, and coins labels all update independently without interfering with each other.

testNewHUDManagerStartsWithDefaults() — Tests that a freshly created HUDManager always starts with Score: 0, Lives: 3, and Coins: 0.

testCoinsLabelIsNotNull() — Tests that the coins label is not null after construction.

testCoinsLabelStartsAtZero() — Tests that the coins label displays "Coins: 0" at the start of the game.

testUpdateCoinsChangesLabel() — Tests that calling updateCoins() correctly updates the coins label text.

testUpdateCoinsMultipleTimes() — Tests that calling updateCoins() multiple times always shows the most recently set value.

testUpdateCoinsResetsToZero() — Tests that the coins label correctly resets back to "Coins: 0" when updateCoins(0) is called.

testCoinsUpdateDoesNotAffectScore() — Tests that updating coins does not change the score label.

testCoinsUpdateDoesNotAffectLives() — Tests that updating coins does not change the lives label.

testMenuPaneIsNotNull() — Tests that the MenuScene pane is not null after construction.

testMenuPaneIsSameReference() — Tests that the pane passed into MenuScene is stored correctly and is the same reference.

testStartButtonIsNotNull() — Tests that the start button is not null after construction.

testMenuVisibleByDefault() — Tests that the menu pane is visible when the game first launches.

testMenuHidesWhenStartFired() — Tests that the menu pane becomes invisible after the start button is fired.

testStartButtonTriggersStartGame() — Tests that firing the start button correctly triggers the startGame action that was set.

testGameNotStartedBeforeButtonClick() — Tests that the game has not started before the start button is clicked.

• ScoreManagerTest

Tests the ScoreManager class, which tracks score, lives, and coins during gameplay and updates the HUDManager labels whenever values change. Also covers the reset() method used when starting a new game.

testScoreStartsAtZero() — Tests that the score is 0 when the game first starts.

testIncrementScoreIncreasesScore() — Tests that calling incrementScore() increases the score by exactly 1.

testIncrementScoreMultipleTimes() — Tests that calling incrementScore() multiple times correctly accumulates the total score.

testScoreUpdatesHUD() — Tests that incrementScore() updates the HUD score label to reflect the new value.

testLivesStartAtThree() — Tests that the player starts with 3 lives.

testLoseLifeDecreasesLives() — Tests that calling loseLife() decreases the lives count by 1.

testLivesDoNotGoBelowZero() — Tests that lives cannot go below 0 even if loseLife() is called more times than there are lives remaining.

testLivesUpdatesHUD() — Tests that loseLife() updates the HUD lives label to reflect the new value.

testCoinsStartAtZero() — Tests that the coin count is 0 when the game first starts.

testAddCoinIncreasesCoins() — Tests that calling addCoin() increases the coin count by 1.

testCoinsUpdatesHUD() — Tests that addCoin() updates the HUD coins label to reflect the new value.

testResetSetsScoreToZero() — Tests that calling reset() sets the score back to 0 after it has been incremented.

testResetSetsLivesToThree() — Tests that calling reset() restores lives back to 3 after a life has been lost.

testResetSetsCoinsToZero() — Tests that calling reset() sets the coin count back to 0 after coins have been collected.

testResetUpdatesAllHUDLabels() — Tests that reset() updates all three HUD labels back to their starting values in a single call.

Unit testing by Paakhi Agrawala:

- ## ObstacleTest

Tests the Obstacle class, which extends GameObject and implements Collidable. Milestone 2 adds theme and size tiers (SMALL/MEDIUM/LARGE with real pixel

dimensions), a manual BoundingBox for collision detection that works from frame 1 without requiring a JavaFX layout pass, scrollUp() for oncoming obstacle behaviour, and an explosion overlay on player hit.

setYUpdatesLogicalYAndSpriteY(): Tests that calling setY() updates both the internal logical Y field and the sprite's rendered Y position on screen. Ensures the ImageView stays in sync with the obstacle's logical position so the visual and collision box are always aligned.

scrollDownMovesObstacleVerticallyByGivenSpeed(): Tests that scrollDown() increases the obstacle's Y coordinate by exactly the given speed value each call, and that the sprite's Y matches. Confirms the frame-by-frame downward movement used by small and medium obstacles and static large obstacles.

isOffscreenReturnsFalseWhenObstacleIsWithinScreenHeight(): Tests that isOffscreen() returns false when the obstacle's Y equals the screen height boundary. Verifies the boundary condition is exclusive — an obstacle exactly at the edge is not yet considered offscreen and should not be respawned.

isOffscreenReturnsTrueWhenObstacleIsPastScreenHeight(): Tests that isOffscreen() returns true when the obstacle's Y exceeds the screen height. This triggers respawn logic in ObstacleManager, repositioning the obstacle above the screen for reuse in the continuous scroll.

fullConstructorUsesCorrectSizeDimensions(): Tests that the full 5-argument constructor sets width and height from the Size enum constants (SMALL: 48×60, MEDIUM: 74×103, LARGE: 140×200) rather than a hardcoded value. Ensures the hit-box dimensions match the actual PNG asset dimensions for accurate collision detection.

getBoundsIsNonEmpty(): Tests that getBounds() returns a BoundingBox with positive width and height. This confirms the manual BoundingBox implementation is producing valid bounds that can be used in intersection checks — a zero-size box would make collisions impossible.

onCollisionWithPlayerSetsFlag(): Tests that calling onCollision() with a Player argument sets the hasCollidedWithPlayer flag to true. This flag is read by

ObstacleManager.checkCollisions(): to trigger the screen shake, red flash, and explosion overlay without double-firing on every subsequent frame.

resetCollisionClearsFlag(): Tests that resetCollision() sets hasCollidedWithPlayer back to false. Called by ObstacleManager after each respawn so the obstacle can register a fresh collision with the player on its next pass.

scrollUpDecreasesY(): Tests that scrollUp() decreases the obstacle's Y coordinate by exactly the given speed value. Used by city and park large obstacles which scroll upward toward the player to simulate oncoming traffic and animals.

- **ObstacleManagerTest**

Tests the ObstacleManager class, which manages all obstacles and coins in the game. Milestone 2 adds time-based theme switching (CITY → PARK → SUBWAY), real PNG sprite loading, manual BoundingBox collision detection, six coin drop patterns, obstacle-to-obstacle gap enforcement, and a difficulty ramp. A test constructor accepting stub images avoids any file I/O during testing.

spawnObstacleAddsObstacleToList(): Tests that spawnObstacle() adds exactly one Obstacle to the internal obstacles list. Verifies the manager is tracking all spawned objects so they can be scrolled, collision-checked, and respawned each frame.

spawnObstacleAddsObstacleSpriteToPane(): Tests that the spawned obstacle's ImageView sprite is added to the JavaFX Pane, making it visible in the game scene. Also confirms the sprite reference in the list matches the node in the pane — they must be the same object.

spawnObstacleCreatesObstacleWithCorrectInitialValues(): Tests that the spawned obstacle's logical X and Y match the values passed to spawnObstacle(). Confirms the centreX/startY parameters are correctly forwarded to the Obstacle constructor so obstacles appear at the intended positions.

checkCollisionsNoHitWhenFarApart(): Tests that checkCollisions() does not flag a collision when the obstacle and player are far apart vertically (obstacle at y=0, player at y=700). Verifies the manual BoundingBox intersection correctly rejects non-overlapping positions.

checkCollisionsNoHitDifferentLane(): Tests that checkCollisions() does not flag a collision when the obstacle and player are in different horizontal lanes (obstacle centred at x=380, player left-edge at x=480, giving a 132px gap). Confirms the x-axis gap check is working correctly.

spawnCoinAddsCoinToList(): Tests that spawnCoin() adds a Coin to the internal coins list when the position does not overlap any obstacle. Verifies coins are tracked by the manager so they can be scrolled and collected each frame.

spawnCoinReturnsNullWhenOverlappingObstacle(): Tests that spawnCoin() returns null when the candidate position overlaps an existing obstacle's bounds. This prevents coins from appearing on top of obstacles, ensuring the player always has a clear path to collect coins without being forced into a collision.

themeIsCityByDefault(): Tests that a freshly constructed ObstacleManager starts with the CITY theme. Confirms the default state matches the first 100 seconds of gameplay where city obstacles should appear.

setElapsedSecondsSwitchesToParkAt100(): Tests that passing 105.0 seconds to setElapsedSeconds() switches the current theme to PARK. Verifies the 100-second threshold is correct and that theme switching is driven by elapsed time rather than frame count.

setElapsedSecondsSwitchesToSubwayAt200(): Tests that passing 205.0 seconds to setElapsedSeconds() switches the current theme to SUBWAY. Verifies the 200-second threshold and confirms the three-theme progression works correctly.

setElapsedSecondsStaysOnSubwayAfter300(): Tests that passing 305.0 seconds keeps the theme on SUBWAY rather than looping back to CITY. Confirms the theme progression is linear and does not cycle — once subway is reached the game stays in that theme for the remainder of the session.

Unit testing by Emmanuel Ayobanjo:

- **SceneryTileTest**

Tests the SceneryTile class, which holds a single ImageView and tracks its position in the scrolling background. Milestone 2 adds self-scrolling behaviour via a scrollSpeed field and an update() method.

testInitialPosition() — Tests that the X and Y coordinates are correctly assigned when a SceneryTile is constructed.

testSetXUpdatesSpriteX() — Tests that calling setX() updates both the internal field and the sprite's rendered X position on screen.

testSetYUpdatesSpriteY() — Tests that calling setY() updates both the internal field and the sprite's rendered Y position on screen.

testGetSpriteNotNull() — Tests that the ImageView sprite is never null after construction — a null sprite would cause a NullPointerException during rendering.

testSetImageSwapsImage() — Tests that setImage() correctly replaces the displayed image on the sprite, which is required for tile recycling.

testGetHeightReturnsImageHeight() — Tests that getHeight() returns a positive value when a valid image is loaded, used by SceneryManager for offscreen detection.

testGetHeightFallbackWhenImageNull() — Tests that getHeight() returns the default fallback value of 192 when the tile's image is null, preventing division-by-zero errors during tile stacking.

testScrollSpeedStoredCorrectly() — Tests that the scrollSpeed value passed to the constructor is stored and retrievable via getScrollSpeed().

testSetScrollSpeedUpdatesSpeed() — Tests that setScrollSpeed() correctly changes the stored scroll rate, allowing speed adjustments during gameplay.

testUpdateIncreasesYByScrollSpeed() — Tests that calling update() moves the tile downward by exactly the scrollSpeed value each frame.

testUpdateSyncsSpriteY() — Tests that after update(), the sprite's rendered Y coordinate matches the tile's logical Y position.

testUpdateDoesNotChangeSpriteX() — Tests that update() only affects the Y axis — the sprite's X coordinate must remain unchanged since scrolling is purely vertical.

testMultipleUpdateCallsAccumulateCorrectly() — Tests that three consecutive update() calls at speed 3.0 move the tile exactly 9 pixels, confirming frame-by-frame accumulation.

testZeroScrollSpeedMeansTileDoesNotMove() — Tests that setting scrollSpeed to 0.0 causes update() to leave the tile's Y position unchanged — validates paused or static tile behaviour.

- ## SceneryManagerTest

Tests the SceneryManager class, which manages the list of SceneryTile objects and drives the vertical scrolling background effect. Milestone 2 adds update()-driven recycling, overlay tile support, and theme switching.

testInitialTileListIsEmpty() — Tests that the tile list starts empty before any tiles are spawned.

testSpawnTileAddsTileToList() — Tests that calling spawnTile() adds exactly one tile to the manager's internal tile list.

testSpawnTileAddsSpriteToPane() — Tests that the spawned tile's sprite is added to the JavaFX Pane, making it visible in the game scene.

testSpawnMultipleTiles() — Tests that spawning three tiles results in exactly three tiles tracked in the list.

testScrollTilesMovesTilesDown() — Tests that calling scrollTiles() moves all tiles to a lower Y position, confirming downward scroll direction.

testScrollTilesMovesDownByScrollSpeed() — Tests that each scrollTiles() call moves tiles by exactly the configured scroll speed value — verifies the scroll rate is precise.

testOffscreenTileIsRecycledToTop() — Tests that a tile which moves below screenHeight is recycled back above the screen (negative Y), implementing the continuous scroll effect.

testTileCountUnchangedAfterRecycle() — Tests that recycling reuses the existing tile object rather than creating a new one — confirms no unnecessary object creation during gameplay.

testGetScrollSpeed() — Tests that getScrollSpeed() returns the correct value set at construction.

testSetScrollSpeed() — Tests that setScrollSpeed() updates the scroll rate correctly, allowing speed changes during gameplay.

testSpawnWithNoVariantsDoesNotCrash() — Tests that spawnTile() handles an empty variants array gracefully without throwing an exception — defensive test for misconfiguration.

testUpdateRecyclesTileWhenOffscreen() — Tests that update() recycles a tile positioned at y=767 back above the screen, confirming that the game-loop-driven update path handles recycling.

testUpdateDoesNotRecycleTileStillOnScreen() — Tests that update() only scrolls a tile that is still within screen bounds without recycling it — prevents premature recycling.

testUpdateScrollsAllTiles() — Tests that a single update() call scrolls every tile in the list by the configured scrollSpeed, not just the first tile.

testInitialOverlayListIsEmpty() — Tests that the overlay tile list starts empty before any building overlays are spawned.

testSpawnOverlayTileAddsToOverlayList() — Tests that calling spawnOverlayTile() adds exactly one tile to the manager's overlay list.

testSpawnOverlayTileAddsSpriteToPane() — Tests that a spawned overlay tile's sprite is added to the JavaFX Pane, making the building visible on screen.

testOverlayTilesScrollAtSameSpeedAsBackground() — Tests that overlay tiles scroll downward at the same speed as background tiles, keeping buildings aligned with the road surface.

testOffscreenOverlayTileIsRecycled() — Tests that an overlay tile which scrolls below the screen is recycled back to a negative Y position above the viewport.

testOverlayCountUnchangedAfterRecycle() — Tests that recycling an overlay tile reuses the same object rather than creating a new one — consistent with the background tile recycling pattern.

testBackgroundAndOverlayTilesAreIndependentLists() — Tests that background tiles and overlay tiles are stored in separate lists — spawning one type does not affect the count of the other.

testSetThemeVariantsChangesVariantArray() — Tests that calling setSceneryVariants() with a new image array allows spawnTile() to work without crashing, confirming the variant swap was accepted.

testRecycledTilePicksUpNewVariantAfterThemeSwitch() — Tests that when the variant array is swapped and a tile is recycled, the recycled tile's image comes from the new variant set — confirms live theme transitions.

testSwitchingToNullVariantsDoesNotCrashOnUpdate() — Tests that setting variants to null does not cause update() to throw an exception — defensive test for mid-game theme transitions.

- **AssetLoaderTest**

Tests the AssetLoader class, which preloads scenery tile images from resources at startup. Milestone 2 adds themed loading via loadThemeAssets() for city, park, and subway themes with 6 tiles each.

testGetSceneryVariantsNullBeforeLoad() — Tests that getSceneryVariants() returns null before loadSceneryAssets() is called, confirming no premature loading occurs.

testLoadSceneryAssetsCreatesArray() — Tests that calling loadSceneryAssets() initialises the variants array so it is no longer null.

testLoadSceneryAssetsReturns16Slots() — Tests that the returned array always contains exactly 16 slots regardless of missing asset files, matching the required scenery variant count.

testNoSlotThrowsException() — Tests that loadSceneryAssets() does not throw an exception when asset files are missing — the fallback logic should handle missing files silently.

testLoadCityAssetsDoesNotThrow() — Tests that loadThemeAssets("city") does not throw an exception, confirming the city asset path resolution works correctly.

testLoadCityAssetsCreatesArray() — Tests that after loading the city theme, getSceneryVariants() returns a non-null array.

testLoadCityAssetsReturns6Slots() — Tests that the city theme produces an array of exactly 6 slots (start, mid_1 through mid_4, and top).

testLoadParkAssetsDoesNotThrow() — Tests that loadThemeAssets("park") does not throw an exception, confirming the park asset path resolution works correctly.

testLoadParkAssetsCreatesArray() — Tests that after loading the park theme, getSceneryVariants() returns a non-null array.

testLoadParkAssetsReturns6Slots() — Tests that the park theme produces an array of exactly 6 slots, matching the standard theme tile count.

testLoadSubwayAssetsDoesNotThrow() — Tests that loadThemeAssets("subway") does not throw an exception, confirming the subway asset path resolution works correctly.

testLoadSubwayAssetsCreatesArray() — Tests that after loading the subway theme, getSceneryVariants() returns a non-null array.

testLoadSubwayAssetsReturns6Slots() — Tests that the subway theme produces an array of exactly 6 slots, matching the standard theme tile count.

testLoadUnknownThemeDoesNotThrow() — Tests that passing an unrecognised theme name does not crash — the loader falls back gracefully.

testLoadUnknownThemeReturnsNonNullArray() — Tests that an unknown theme still produces a non-null array, preventing NullPointerExceptions downstream.

testSwitchingThemeOverwritesPreviousArray() — Tests that loading a second theme replaces the first theme's array — getSceneryVariants() reflects the most recently loaded theme.

testLoadThemeAssetsNullThemeDoesNotThrow() — Tests that passing null as the theme name is handled gracefully without throwing an exception.

- **BuildingTileTest**

Tests the BuildingTile class, which extends SceneryTile and adds Collidable behaviour for the side building overlays. Verifies inheritance from SceneryTile, implementation of the Collidable interface, and the architectural rule that plain SceneryTile does not implement Collidable.

testBuildingTileIsInstanceOfSceneryTile() — Tests that BuildingTile is a subclass of SceneryTile, allowing it to be stored in the same tile list as background tiles.

testBuildingTileIsInstanceOfGameObject() — Tests that BuildingTile is a subclass of GameObject, so it can participate in the polymorphic update() loop in GameScene.

testInheritedInitialPosition() — Tests that the X and Y coordinates set via the constructor are correctly inherited from SceneryTile.

testInheritedScrollSpeedStored() — Tests that the scrollSpeed passed to the constructor is stored correctly via the inherited SceneryTile field.

testInheritedUpdateScrollsTileDown() — Tests that calling update() on a BuildingTile moves it downward by the scroll speed, reusing SceneryTile's scrolling logic.

testInheritedUpdateSyncsSpriteY() — Tests that after update(), the BuildingTile's sprite Y coordinate matches its logical Y, inherited from SceneryTile.

testInheritedGetSpriteNotNull() — Tests that the sprite is not null after construction, inherited from SceneryTile.

testInheritedSetImageSwapsImage() — Tests that setImage() correctly swaps the displayed image on the sprite, inherited from SceneryTile.

testBuildingTileImplementsCollidable() — Tests that BuildingTile implements the Collidable interface, so the GameScene collision loop can detect it.

testGetBoundsReturnsNonNull() — Tests that getBounds() returns a non-null Bounds object, required for intersection checks in collision detection.

testOnCollisionDoesNotThrow() — Tests that calling onCollision() with another Collidable does not throw an exception.

testOnCollisionWithNullDoesNotThrow() — Tests that passing null to onCollision() is handled safely without throwing a NullPointerException.

testUpdatePositionSyncsSpriteToLogicalCoords() — Tests that updatePosition() synchronises the sprite's X and Y to the tile's logical coordinates after manual position changes.

testPlainSceneryTileIsNotCollidable() — Tests that a plain SceneryTile does not implement Collidable — only BuildingTile should, confirming MC's architecture decision that the road surface does not trigger collisions.

- **PlayerBuildingCollisionTest**

Tests Player collision with side BuildingTiles and the playable-area road boundaries. Covers road min/max clamping, player snap behaviour on building collision, and sprite synchronisation after collision resolution.

testRoadMinXIsOverlayWidth() — Tests that the road's left boundary equals the overlay column width, defining where the playable area begins.

testRoadMaxXIsScreenWidthMinusOverlayWidth() — Tests that the road's right boundary equals the screen width minus the overlay column width, defining where the playable area ends.

testPlayerCannotMoveLeftOfRoadBoundary() — Tests that after calling updatePosition(), the player's X is clamped so it never goes left of the road's left edge.

testPlayerCannotMoveRightOfRoadBoundary() — Tests that after calling updatePosition(), the player's right edge is clamped so it never exceeds the road's right edge.

testPlayerStartsInsideRoadBoundaries() — Tests that the player's initial position falls within the road boundaries on both sides.

testPlayerCollisionWithLeftBuildingDoesNotThrow() — Tests that calling onCollision() with the left BuildingTile does not throw an exception.

testPlayerSnapsToRoadMinAfterLeftBuildingCollision() — Tests that when the player drifts into the left building, onCollision() snaps the player's X back to roadMinX.

testPlayerSpriteXMatchesLogicalXAfterLeftCollision() — Tests that the sprite's X coordinate stays in sync with the logical X after a left-building collision snap.

testPlayerCollisionWithRightBuildingDoesNotThrow() — Tests that calling onCollision() with the right BuildingTile does not throw an exception.

testPlayerSnapsToRoadMaxAfterRightBuildingCollision() — Tests that when the player drifts into the right building, onCollision() snaps the player's right edge back to roadMaxX.

testPlayerSpriteXMatchesLogicalXAfterRightCollision() — Tests that the sprite's X coordinate stays in sync with the logical X after a right-building collision snap.

testBuildingTileOnCollisionWithPlayerDoesNotThrow() — Tests that calling onCollision() on a BuildingTile with a Player argument does not throw an exception.

testBuildingTileOnCollisionWithNullDoesNotThrow() — Tests that calling onCollision() on a BuildingTile with null is handled safely without a NullPointerException.

- ## GameSceneTest (Overlay Integration)

Tests for the overlay and BuildingTile integration within GameScene. These tests verify that the building overlay system initialises correctly, that BuildingTile objects are present in the game object list, and that overlay spawning respects the minimum gap rule.

testBuildingTilesExistInGameObjects() — Tests that after initializeGameObjects(), at least one BuildingTile is present in the game object list — confirms the overlay setup pipeline is wired into GameScene.

testOverlayTilesExistInSceneryManager() — Tests that the SceneryManager's overlay list is not empty after initialisation, confirming that registerOverlayTile() is being called during setupOverlays().

testOverlayTilesAreNotClustered() — Tests that no two left-side overlay tiles share the same Y position, enforcing the MIN_GAP rule that prevents buildings from stacking on adjacent road tiles.

testBuildingTilesAreCollidable() — Tests that every BuildingTile in the game object list implements Collidable, ensuring the collision detection loop will pick them up.

testOverlayCountIsWithinReasonableBounds() — Tests that the overlay count falls within a reasonable range (2–20), accounting for the randomised spawn chance while guarding against clustering bugs or runaway spawning.

Unit testing by Deliwe Mufute:

- # PlayerTest

Tests the Player class, which represents the player character in Campus Craze. Milestone 2 upgrades the Player to extend GameObject and implement Collidable, replacing the fixed-speed direct movement from Milestone 1 with a velocity-based smooth movement system. Tests cover initial state, movement in all four directions, smooth velocity building, velocity capping, stop command deceleration, road boundary clamping, sprite synchronisation, collision response with BuildingTile and Obstacle, and the polymorphic update() method.

- **Group 1 — Initial State**

testInitialPosition()

Tests that the player's X and Y coordinates match the startX and startY values passed to the constructor. Verifies the GameObject superclass correctly stores the initial position.

testInitialVelocityIsZero()

Tests that both velocityX and velocityY are exactly zero when the player is first created. A non-zero starting velocity would cause the player to drift immediately on game start.

testRoadBoundariesSetCorrectly()

Tests that getRoadMinX() returns the roadLeft value and getRoadMaxX() returns the roadRight value passed to the constructor. These boundaries are set from GameScene's ROAD_LEFT_X and ROAD_RIGHT_X constants.

- **Group 2 — Movement Direction Tests**

testMoveLeftDecreasesX()

Tests that calling moveLeft() followed by updatePosition() results in a lower X value. Confirms the horizontal velocity system moves the player in the correct direction when the left key is pressed.

testMoveRightIncreasesX()

Tests that calling moveRight() followed by updatePosition() results in a higher X value. Confirms the horizontal velocity system moves the player in the correct direction when the right key is pressed.

testMoveUpDecreasesY()

Tests that calling moveUp() followed by updatePosition() results in a lower Y value. In JavaFX coordinates, lower Y means higher on screen, representing forward movement. Confirms the Y-axis velocity system works correctly.

testMoveDownIncreasesY()

Tests that calling moveDown() followed by updatePosition() results in a higher Y value, representing slight backward movement. Confirms the Y-axis velocity system works in both directions.

- **Group 3 — Smooth Velocity Building**

testVelocityBuildsOverMultipleFrames()

Tests that holding a key for two consecutive frames produces a higher velocity on frame 2 than frame 1. This confirms smooth acceleration rather than instant full-speed movement — a key Milestone 2 requirement for natural-feeling player control.

- **Group 4 — Velocity Cap Tests**

testVelocityCappedAtMaxSpeedX()

Tests that holding the right key for 50 consecutive frames never causes velocityX to exceed MAX_SPEED_X (6.0). Without this cap, the player would accelerate indefinitely and eventually fly off screen.

testVelocityCappedAtMaxSpeedY()

Tests that holding the down key for 50 consecutive frames never causes velocityY to exceed MAX_SPEED_Y (3.0). The vertical cap is intentionally smaller than the horizontal cap to keep forward/backward movement subtle as required.

- **Group 5 — Stop Command Deceleration Tests**

testStopLeftCausesDeceleration()

Tests that after stopLeft() is called, the player decelerates and travels less than 10 pixels over the next 20 frames. Verifies the DECELERATION friction factor (0.85) brings the player to a smooth stop rather than an abrupt halt.

testStopRightCausesDeceleration()

Tests the same deceleration behaviour for the right direction. After stopRight(), the player should coast to a near-stop within 20 frames, confirming friction applies symmetrically.

testStopUpCausesDeceleration()

Tests that after stopUp() is called, vertical velocity decelerates smoothly over 20 frames. Confirms the same friction system applies to the Y-axis as the X-axis.

testStopDownCausesDeceleration()

Tests that after stopDown() is called, downward vertical velocity decelerates smoothly. Confirms complete stop command coverage for all four movement directions.

- **Group 6 — Road Boundary Constraint Tests**

testCannotMoveLeftOfRoadBoundary()

Tests that pushing the player against the left road boundary for 20 frames never results in a position below ROAD_LEFT_X. Verifies that updatePosition() enforces the left boundary by clamping X and zeroing velocityX on contact.

testCannotMoveRightOfRoadBoundary()

Tests that pushing the player against the right road boundary for 20 frames never results in the player's right edge exceeding ROAD_RIGHT_X. Verifies right boundary clamping including the width offset.

testCannotMoveAboveTopBoundary()

Tests that pushing up for 100 frames never takes the player above minY (START_Y - 80). Verifies the top Y boundary is enforced, limiting how far forward the player can move.

testCannotMoveBelowBottomBoundary()

Tests that pushing down for 100 frames never takes the player below maxY (START_Y + 40). Verifies the bottom Y boundary is enforced, limiting backward movement to a slight range.

- **Group 7 — Sprite Sync Test**

testSpriteSyncsWithPositionAfterUpdate()

Tests that after setting x and y directly and calling updatePosition(), the ImageView sprite's X and Y coordinates match the player's logical position. This is critical because JavaFX renders the sprite, not the logical coordinates — they must always stay in sync.

- **Group 8 — Collision with BuildingTile**

testCollisionWithLeftBuildingSnapsToRoadMin()

Tests that when the player's X is set to ROAD_LEFT - 10 (inside the left building) and onCollision() is called with a left BuildingTile, the player's X snaps back to exactly ROAD_LEFT. Verifies the snap-to-boundary collision response.

testCollisionWithRightBuildingSnapsToRoadMax()

Tests that when the player drifts into the right building and onCollision() is called with a right BuildingTile, the player's X snaps so the right edge aligns with ROAD_RIGHT. Verifies the right-side boundary snap.

testSpriteSyncsAfterBuildingCollision()

Tests that after a building collision snap, the sprite's X coordinate matches the updated logical X. Ensures the visual position is correct after collision resolution — prevents the sprite appearing inside the building.

testOnCollisionWithNullDoesNotThrow()

Tests that calling onCollision(null) is handled safely without throwing a NullPointerException. The method includes a null guard at the top which this test verifies is working.

- **Group 9 — Collision with Obstacle**

testCollisionWithObstacleReducesLives()

Tests that calling onCollision() with an Obstacle does not throw an exception when scoreManager is null. The null check on scoreManager prevents a crash when the player has not yet been wired to a ScoreManager — important for isolated unit testing.

- **Group 10 — Polymorphic update() Method**

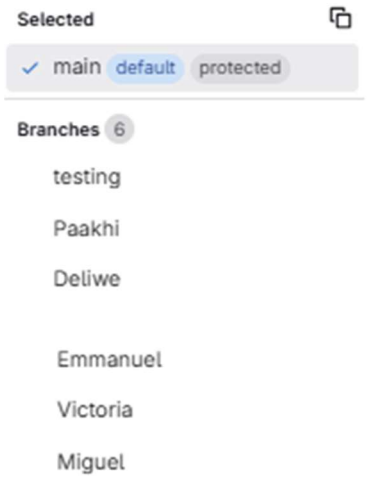
testUpdateMethodMovesPlayer()

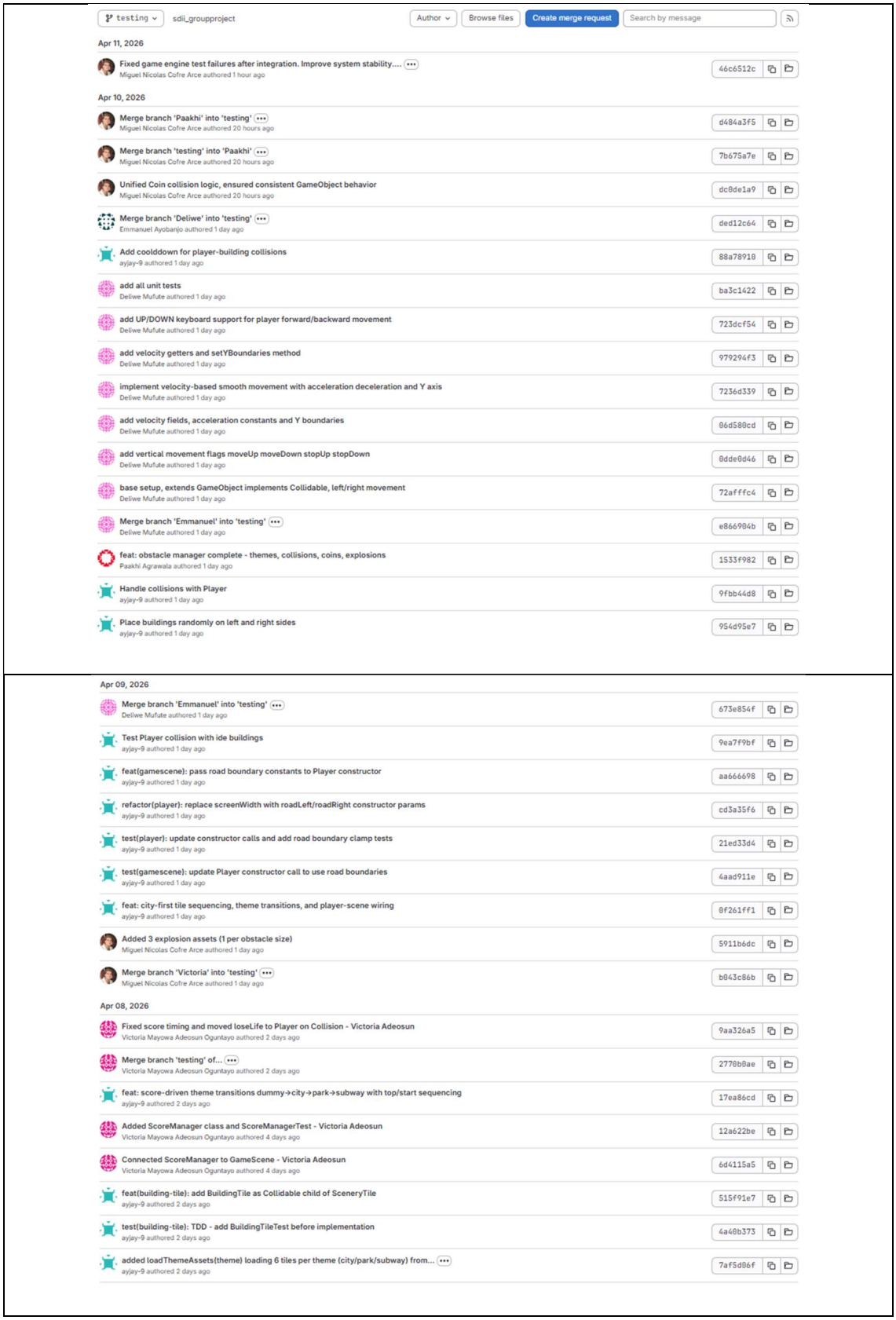
Tests that calling update() — the polymorphic method GameScene calls on every GameObject each frame — moves the player when moveRight() is active. Confirms the delegation chain works: GameScene calls update(), Player.update() calls updatePosition(), movement occurs.

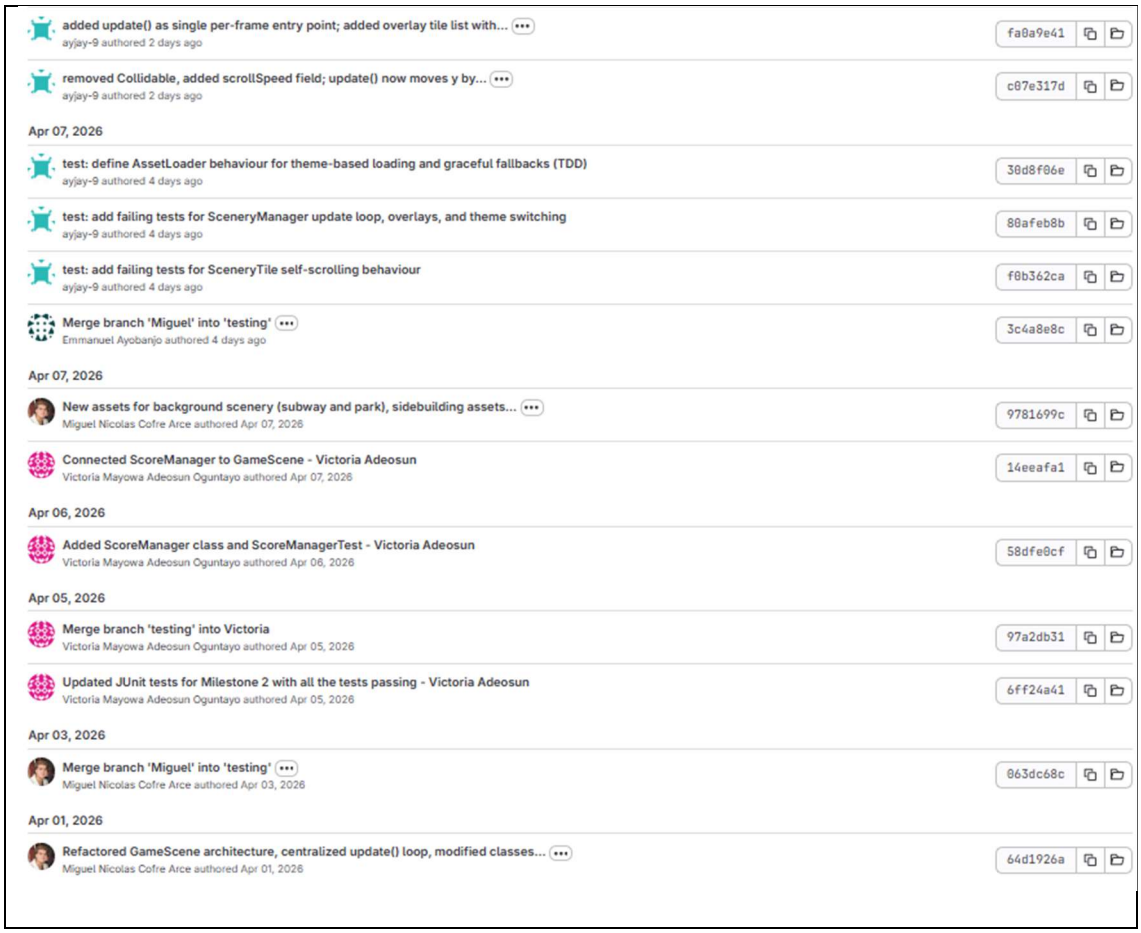
testUpdateDelegatesToUpdatePosition()

Tests that two identical players produce the same X position after one calls update() and the other calls updatePosition() directly. Confirms update() is a pure delegate to updatePosition() with no additional side effects.

3.8 Commit Logs & Branches Tree

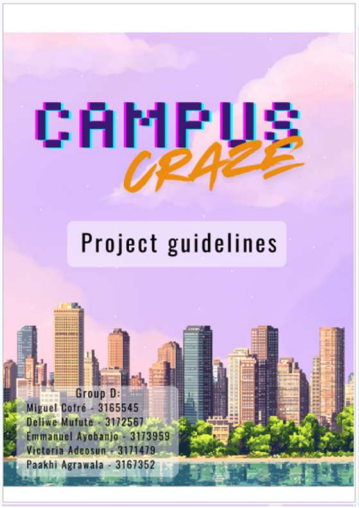
Branches in repository:	
Commit Logs	

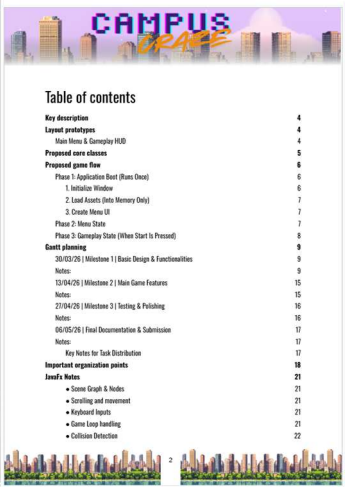

testing
sdii_groupproject
Author
Browse files
Create merge request
Search by message
Apr 11, 2026
Fixed game engine test failures after integration. Improve system stability....
Miguel Nicolas Cofre Arce authored 1 hour ago
46c6512c
Apr 10, 2026
Merge branch 'Paakhi' into 'testing'
Miguel Nicolas Cofre Arce authored 20 hours ago
d484a3f5
Merge branch 'testing' into 'Paakhi'
Miguel Nicolas Cofre Arce authored 20 hours ago
7b675a7e
Unified Coin collision logic, ensured consistent GameObject behavior
Miguel Nicolas Cofre Arce authored 20 hours ago
dc0de1a9
Merge branch 'Deliwe' into 'testing'
Emmanuel Ayobanjo authored 1 day ago
ded12c64
Add cooldown for player-building collisions
ayjay-9 authored 1 day ago
88a78910
add all unit tests
Deliwe Mufute authored 1 day ago
ba3c1422
add UP/DOWN keyboard support for player forward/backward movement
Deliwe Mufute authored 1 day ago
723dcf54
add velocity getters and setYBoundaries method
Deliwe Mufute authored 1 day ago
979294f3
implement velocity-based smooth movement with acceleration deceleration and Y axis
Deliwe Mufute authored 1 day ago
7236d339
add velocity fields, acceleration constants and Y boundaries
Deliwe Mufute authored 1 day ago
06d580cd
add vertical movement flags moveUp moveDown stopUp stopDown
Deliwe Mufute authored 1 day ago
0dde0d46
base setup, extends GameObject implements Collidable, left/right movement
Deliwe Mufute authored 1 day ago
72afffc4
Merge branch 'Emmanuel' into 'testing'
Deliwe Mufute authored 1 day ago
e866904b
feat: obstacle manager complete - themes, collisions, coins, explosions
Paakhi Agrawala authored 1 day ago
1533f982
Handle collisions with Player
ayjay-9 authored 1 day ago
9fbb44d8
Place buildings randomly on left and right sides
ayjay-9 authored 1 day ago
954d95e7
Apr 09, 2026
Merge branch 'Emmanuel' into 'testing'
Deliwe Mufute authored 1 day ago
673e854f
Test Player collision with ide buildings
ayjay-9 authored 1 day ago
9ea7f9bf
feat(gamescene): pass road boundary constants to Player constructor
ayjay-9 authored 1 day ago
aa666698
refactor(player): replace screenWidth with roadLeft/roadRight constructor params
ayjay-9 authored 1 day ago
cd3a35f6
test(player): update constructor calls and add road boundary clamp tests
ayjay-9 authored 1 day ago
21ed33d4
test(gamescene): update Player constructor call to use road boundaries
ayjay-9 authored 1 day ago
4aad911e
feat: city-first tile sequencing, theme transitions, and player-scene wiring
ayjay-9 authored 1 day ago
0f261ff1
Added 3 explosion assets (1 per obstacle size)
Miguel Nicolas Cofre Arce authored 1 day ago
5911b6dc
Merge branch 'Victoria' into 'testing'
Miguel Nicolas Cofre Arce authored 1 day ago
b043c86b
Apr 08, 2026
Fixed score timing and moved loseLife to Player on Collision - Victoria Adeosun
Victoria Mayowa Adeosun Oguntayo authored 2 days ago
9aa326a5
Merge branch 'testing' of...
Victoria Mayowa Adeosun Oguntayo authored 2 days ago
2770b0ae
feat: score-driven theme transitions dummy→city→park→subway with top/start sequencing
ayjay-9 authored 2 days ago
17ea86cd
Added ScoreManager class and ScoreManagerTest - Victoria Adeosun
Victoria Mayowa Adeosun Oguntayo authored 4 days ago
12a622be
Connected ScoreManager to GameScene - Victoria Adeosun
Victoria Mayowa Adeosun Oguntayo authored 4 days ago
6d4115a5
feat(building-tile): add BuildingTile as Collidable child of SceneryTile
ayjay-9 authored 2 days ago
515f91e7
test(building-tile): TDD - add BuildingTileTest before implementation
ayjay-9 authored 2 days ago
4a40b373
added loadThemeAssets(theme) loading 6 tiles per theme (city/park/subway) from...
ayjay-9 authored 2 days ago
7af5d86f

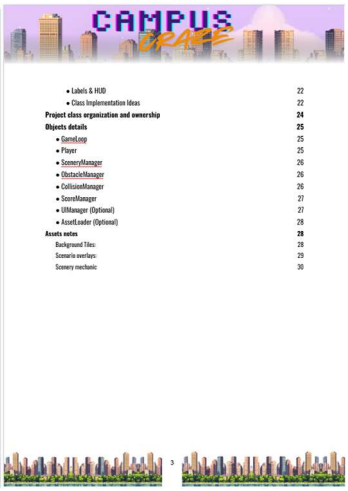

added update() as single per-frame entry point; added overlay tile list with...
ayjay-9 authored 2 days ago
fa0a9e41
removed Collidable, added scrollSpeed field; update() now moves y by...
ayjay-9 authored 2 days ago
c07e317d
Apr 07, 2026
test: define AssetLoader behaviour for theme-based loading and graceful fallbacks (TDD)
ayjay-9 authored 4 days ago
30d8f06e
test: add failing tests for SceneryManager update loop, overlays, and theme switching
ayjay-9 authored 4 days ago
80afeb8b
test: add failing tests for SceneryTile self-scrolling behaviour
ayjay-9 authored 4 days ago
f0b362ca
Merge branch 'Miguel' into 'testing'
Emmanuel Ayobanjo authored 4 days ago
3c4a8e8c
Apr 07, 2026
New assets for background scenery (subway and park), sidebuilding assets...
Miguel Nicolas Cofre Arce authored Apr 07, 2026
9781699c
Connected ScoreManager to GameScene - Victoria Adeosun
Victoria Mayowa Adeosun Oguntayo authored Apr 07, 2026
14eeafa1
Apr 06, 2026
Added ScoreManager class and ScoreManagerTest - Victoria Adeosun
Victoria Mayowa Adeosun Oguntayo authored Apr 06, 2026
58dfe0cf
Apr 05, 2026
Merge branch 'testing' into Victoria
Victoria Mayowa Adeosun Oguntayo authored Apr 05, 2026
97a2db31
Updated JUnit tests for Milestone 2 with all the tests passing - Victoria Adeosun
Victoria Mayowa Adeosun Oguntayo authored Apr 05, 2026
6ff24a41
Apr 03, 2026
Merge branch 'Miguel' into 'testing'
Miguel Nicolas Cofre Arce authored Apr 03, 2026
863dc68c
Apr 01, 2026
Refactored GameScene architecture, centralized update() loop, modified classes...
Miguel Nicolas Cofre Arce authored Apr 01, 2026
64d1926a

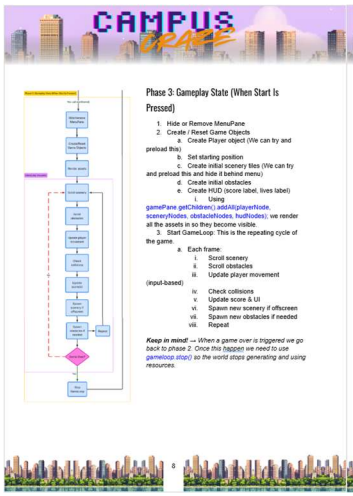
4. Appendix:

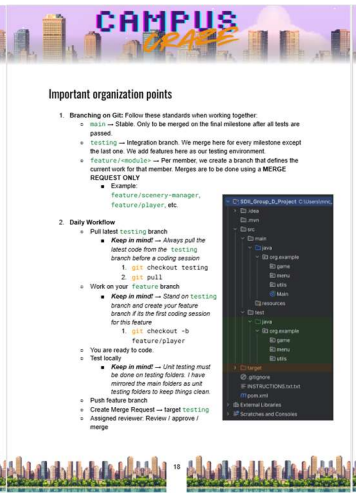
An additional document has been worked on through the development of the game and has been appended to the delivery (Extraction preview below):

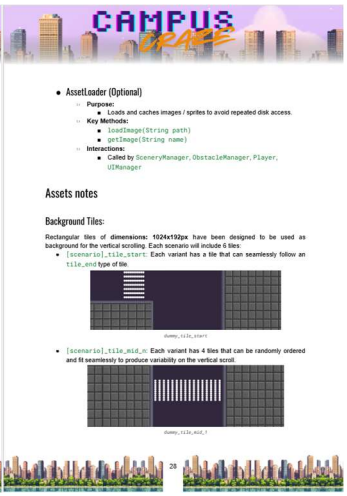


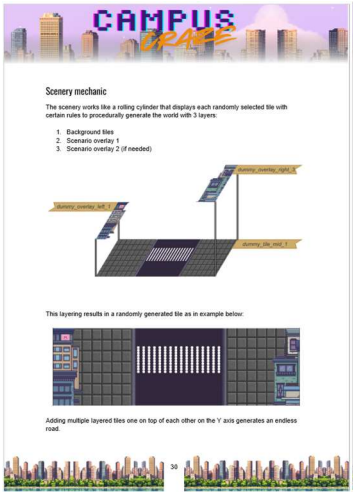


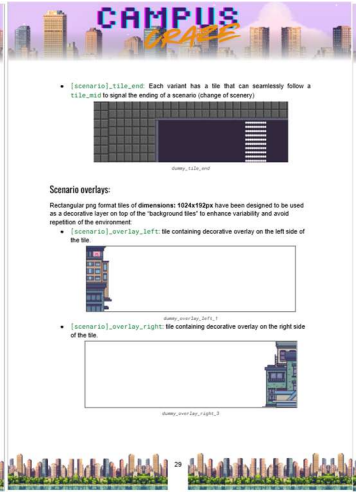


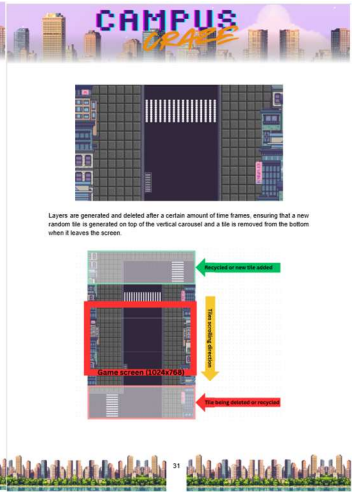












5. References

1.- Sakaue, S., Kimura, T., & Nishino, H. (2023). Reducing Objective Difficulty Without Influencing Subjective Difficulty in a Video Game. *Proceedings of the 5th ACM International Conference on Multimedia in Asia*. https://doi.org/10.1145/3595916.3626361.